



DISTRIBUTED RATE LIMITER

Complete Build Guide

Build a deterministic token bucket service with a three node Raft lite consensus layer, persistent storage, secure APIs, an admin dashboard, observability, fault tests, and deployment.

Audience

Motivated beginners using Claude Code, Codex, or ChatGPT

Primary stack

Go 1.26.5, bbolt 1.5.0, Docker, Prometheus, Caddy

Guide version

1.0 | July 2026

Important: this is an educational Raft implementation with fixed membership. It is designed to demonstrate consensus mechanics and failure handling, not replace a formally verified production consensus library.

Table of Contents

0. Assumptions and Scope	11
0.1 Fixed project decisions	11
0.2 What Raft lite means here	11
0.3 Development and deployment assumptions	12
0.4 Definition of complete	12
1. Project Overview	13
1.1 What the project is	13
1.2 The problem it solves	13
1.3 Intended users	14
1.4 Main features	14
1.5 Skills demonstrated	14
1.6 Terms used throughout the guide	15
2. Final Product Layout	15
2.1 How a user reaches the product	15
2.2 Screen inventory	16
2.3 Screen details	16
2.3.1 Admin sign in	16
2.3.2 Overview dashboard	17
2.3.3 Policies page	17
2.3.4 API clients page	18
2.3.5 Decisions page	18
2.3.6 Cluster page	18
2.3.7 API examples page	18
2.4 Important user flows	18
Client decision flow	18
Administrator setup flow	19
Leader failure flow	19
3. Technical Architecture	19
3.1 Version locked technology stack	20
3.2 Front end architecture	20
3.3 Back end architecture	20
3.4 Raft state and rules	21
3.5 Deterministic token bucket	21
3.6 Database choice and write ownership	22

3.7 APIs and external services	22
3.8 Authentication and security	22
3.9 Hosting and deployment	23
3.10 Testing architecture	23
3.11 Known limitations	23
4. Prerequisites and Setup	24
4.1 Required hardware, software, and accounts	24
4.2 Knowledge expected	24
4.3 Install base tools on Ubuntu	24
4.4 Install Go 1.26.5	24
4.5 Install Docker Engine and Compose	25
4.6 Install Go quality tools	25
4.7 Create the repository	25
4.8 Create safe development secrets	26
4.9 Initial verification	26
5. Project Folder Structure	26
5.1 Complete recommended tree	27
5.2 Folder responsibilities	29
5.3 Important file rules	29
6. Complete Development Plan	29
6.1 Phase 1: Bootstrap, configuration, and process lifecycle	30
Objective	30
Why this phase is necessary	30
Exact actions	30
Commands	30
Files to create or modify	31
What the AI coding agent should produce	31
How to test	31
Expected output	31
Common errors and corrections	31
6.2 Phase 2: bbolt storage and migrations	32
Objective	32
Why this phase is necessary	32
Exact actions	32
Commands	32
Files to create or modify	32
What the AI coding agent should produce	32
How to test	32

Expected output	33
Common errors and corrections	33
6.3 Phase 3: Deterministic token bucket state machine	33
Objective	33
Why this phase is necessary	33
Exact actions	33
Commands	34
Files to create or modify	34
What the AI coding agent should produce	34
How to test	34
Expected output	34
Common errors and corrections	34
6.4 Phase 4: Raft types, persistent transitions, and event loop	34
Objective	34
Why this phase is necessary	34
Exact actions	35
Commands	35
Files to create or modify	35
What the AI coding agent should produce	35
How to test	35
Expected output	35
Common errors and corrections	35
6.5 Phase 5: Leader election and RequestVote	36
Objective	36
Why this phase is necessary	36
Exact actions	36
Commands	36
Files to create or modify	36
What the AI coding agent should produce	36
How to test	36
Expected output	36
Common errors and corrections	37
6.6 Phase 6: Log replication, quorum commit, and ordered apply	37
Objective	37
Why this phase is necessary	37
Exact actions	37
Commands	37
Files to create or modify	38
What the AI coding agent should produce	38
How to test	38
Expected output	38
Common errors and corrections	38

6.7 Phase 7: Public decision API and leader forwarding	38
Objective	38
Why this phase is necessary	38
Exact actions	38
Commands	39
Files to create or modify	39
What the AI coding agent should produce	39
How to test	39
Expected output	39
Common errors and corrections	39
6.8 Phase 8: Admin authentication and secure mutation APIs	40
Objective	40
Why this phase is necessary	40
Exact actions	40
Commands	40
Files to create or modify	40
What the AI coding agent should produce	40
How to test	41
Expected output	41
Common errors and corrections	41
6.9 Phase 9: Admin dashboard and responsive interface	41
Objective	41
Why this phase is necessary	41
Exact actions	41
Commands	42
Files to create or modify	42
What the AI coding agent should produce	42
How to test	42
Expected output	42
Common errors and corrections	42
6.10 Phase 10: Health, readiness, logging, and Prometheus metrics	42
Objective	42
Why this phase is necessary	42
Exact actions	42
Commands	43
Files to create or modify	43
What the AI coding agent should produce	43
How to test	43
Expected output	43
Common errors and corrections	43
6.11 Phase 11: Docker image and three node Compose cluster	43
Objective	43

Why this phase is necessary	44
Exact actions	44
Commands	44
Files to create or modify	44
What the AI coding agent should produce	44
How to test	44
Expected output	45
Common errors and corrections	45
6.12 Phase 12: Integration tests, partitions, restart, and failover scripts	45
Objective	45
Why this phase is necessary	45
Exact actions	45
Commands	45
Files to create or modify	46
What the AI coding agent should produce	46
How to test	46
Expected output	46
Common errors and corrections	46
6.13 Phase 13: Load, performance, security, and quality gates	46
Objective	46
Why this phase is necessary	46
Exact actions	46
Commands	47
Files to create or modify	47
What the AI coding agent should produce	47
How to test	47
Expected output	47
Common errors and corrections	47
6.14 Phase 14: Production style deployment	47
Objective	47
Why this phase is necessary	48
Exact actions	48
Commands	48
Files to create or modify	48
What the AI coding agent should produce	49
How to test	49
Expected output	49
Common errors and corrections	49
6.15 Phase 15: GitHub packaging and reviewer handoff	49
Objective	49
Why this phase is necessary	49
Exact actions	49

Commands	50
Files to create or modify	50
What the AI coding agent should produce	50
How to test	50
Expected output	50
Common errors and corrections	50

7. AI Coding Agent Prompts 50

7.1 Prompt for Phase 1	51
7.2 Prompt for Phase 2	52
7.3 Prompt for Phase 3	53
7.4 Prompt for Phase 4	53
7.5 Prompt for Phase 5	54
7.6 Prompt for Phase 6	55
7.7 Prompt for Phase 7	55
7.8 Prompt for Phase 8	56
7.9 Prompt for Phase 9	56
7.10 Prompt for Phase 10	57
7.11 Prompt for Phase 11	57
7.12 Prompt for Phase 12	58
7.13 Prompt for Phase 13	58
7.14 Prompt for Phase 14	59
7.15 Prompt for Phase 15	59

8. Database and API Details 59

8.1 Database overview	60
8.2 Bucket schema	60
8.3 Raft metadata and log entry	61
8.4 Policy model	61
8.5 API client model	62
8.6 Token bucket state	63
8.7 Decision and idempotency record	63
8.8 Audit event	64
8.9 Migration instructions	64
8.10 API conventions	64
8.11 Endpoint inventory	65
8.12 Health and metrics endpoints	66
8.13 Public decision endpoint	66
Request	66
Allowed response	66

Denied response	67
Error responses	67
8.14 Admin sign in and session endpoints	67
8.15 Policy administration API	67
List policies	67
Create policy	68
Update policy	68
Change activation	68
8.16 Client administration API	68
List clients	68
Create client	68
Revoke client	69
8.17 Audit, overview, cluster, and admin test APIs	69
8.18 RequestVote RPC	69
8.19 AppendEntries RPC	70
8.20 Internal status	70

9. Testing and Quality Checks 70

9.1 Test strategy	71
9.2 Unit test cases	71
9.3 Integration test cases	72
9.4 Invariant checks	72
9.5 API and error handling checks	72
9.6 Security checks	73
9.7 Race, leak, and shutdown checks	73
9.8 Responsive and accessibility checks	73
9.9 Load test cases	74
9.10 Continuous integration checks	74
9.11 Final completion checklist	75
Repository and setup	75
Token bucket	75
Raft lite	75
API and security	75
Dashboard	76
Testing and operations	76
Documentation	76

10. Deployment 77

10.1 Deployment choices	77
10.2 Required production environment variables	77
10.3 Production image	77

10.4 Single server Compose design	78
10.5 Caddy production configuration	78
10.6 Server provisioning	79
10.7 First deployment verification	79
10.8 Production security checks	80
10.9 Back up a follower without losing quorum	80
10.10 Restore rehearsal	80
10.11 Upgrade and rollback	81
10.12 Three virtual machine variant	81
10.13 Live acceptance checklist	81

11. GitHub Documentation 82

11.1 Repository organization	82
11.2 Recommended README structure	82
Title and one sentence	82
Status badges	83
Demonstration	83
Why this project exists	83
Features	83
Architecture	83
Consistency behavior	83
Quick start	83
API example	83
Tests	83
Metrics and operations	84
Deployment	84
Design decisions	84
Limitations	84
Roadmap, contributing, and license	84
11.3 Screenshot plan	84
11.4 CI workflow outline	84
11.5 Publishing safely	85
11.6 Reviewer oriented documentation	85

12. Troubleshooting Guide 86

12.1 Installation and environment	86
12.2 Configuration and startup	86
12.3 Leader election	87
12.4 Replication and commit	87
12.5 Token bucket and idempotency	87
12.6 API and authentication	88

12.7 Dashboard	88
12.8 Docker and networking	89
12.9 Deployment and HTTPS	89
12.10 Tests and CI	89
13. Future Improvements	90
13.1 Essential requirements for version 1.0	90
13.2 High value next improvements	90
Snapshot and log compaction	90
Linearizable read path	90
Mutual TLS between nodes	90
Clock monitoring	90
Snapshot aware backup	90
13.3 Advanced distributed systems extensions	91
Dynamic membership	91
Pre vote and leadership transfer	91
Read leases	91
Batched proposals and group commit	91
Sharding	91
13.4 Product improvements	91
13.5 Research and verification improvements	91
Appendix A. Command Reference	91
A.1 Daily development	92
A.2 Local cluster	92
A.3 Diagnose nodes	92
A.4 Final release check	92
Appendix B. Environment Variable Reference	92
Appendix C. Interview Demonstration Script	93
Appendix D. Official References and Version Notes	93
Appendix E. Final Handoff	94

0. Assumptions and Scope

This guide turns the earlier project idea into one specific build. The choices below are deliberate so that a beginner and an AI coding agent can work from the same contract. If you change a choice, update the architecture document, tests, environment example, Docker files, and every agent prompt before continuing.

0.1 Fixed project decisions

Decision	Chosen approach	Reason
Project	Distributed rate limiter	It has a smaller and clearer correctness surface than a job queue while still demonstrating distributed systems skills.
Limiting algorithm	Deterministic token bucket	It supports bursts, steady refill, variable request cost, and precise unit tests.
Consensus	Raft lite written from scratch	The project must demonstrate leader election, replicated logs, quorum commit, and recovery without importing a Raft library.
Cluster size	Three fixed nodes	Three nodes can tolerate one unavailable node while keeping the implementation understandable.
Language	Go 1.26.5	Go has strong concurrency primitives, a useful standard library, fast builds, and a good fit for network services.
Node storage	bbolt 1.5.0	Each node needs a durable local log and state. bbolt is an embedded Go key value database, so no separate database server is required.
Front end	Go templates, HTML, CSS, and small vanilla JavaScript files	A separate JavaScript framework would add build complexity without strengthening the distributed systems story.
Metrics	Prometheus client 1.24.1 and Prometheus 3.13.1 LTS	These provide inspectable service and consensus metrics with a stable monitoring path.
Local orchestration	Docker Engine 29.6.1 and Docker Compose 5.3.0	Containers make three isolated nodes and persistent volumes reproducible on one computer.
Public gateway	Caddy 2.11.4	Caddy supplies simple reverse proxying and automatic HTTPS in production.
Load testing	k6 2.1.0	It can generate repeatable concurrent traffic and enforce performance thresholds.
Linting	golangci-lint 2.12.2	It catches common Go problems in one repeatable command.
Vulnerability scan	govulncheck 1.1.4	It reports known vulnerabilities that are reachable from Go code.

0.2 What Raft lite means here

Raft is a consensus algorithm that lets several machines agree on one ordered log of commands. The leader receives a command, copies it to followers, and commits it after a majority stores it. Every node applies committed commands in the same order to a deterministic state machine.

This project implements the following essential Raft behavior:

- Persistent current term and vote.
- Follower, candidate, and leader states.
- Randomized election timeouts.
- RequestVote remote procedure calls.
- AppendEntries heartbeats and log replication.
- Log consistency checks using previous index and previous term.
- Per follower next index and match index tracking.
- Majority commit, including the current term commit rule.
- Ordered state machine application.
- Leader discovery and safe client retry behavior.
- Durable restart and log replay.

The first complete version intentionally excludes dynamic membership, joint consensus, snapshots, log compaction, leadership transfer, pre vote, read index, and formal verification. These are listed as future work. The log can grow, so the deployment section includes backup and size monitoring. This boundary should be stated honestly in the README and interviews.

Safety boundary

Do not describe this service as production ready Raft. It is an educational implementation that follows core Raft safety rules under a fixed three node configuration. A production system should use a mature and audited consensus library.

0.3 Development and deployment assumptions

The exact commands target Ubuntu 24.04 LTS on an x86 64 computer or inside Windows Subsystem for Linux 2. macOS users can use Docker Desktop and the official Go package, but should run the same repository commands after installation. The guide assumes at least 8 GB of memory, 15 GB of free disk space, Git, a terminal, and permission to run Docker.

Local development runs three containers on one machine. These are independent processes with independent data volumes, so leader election, replication, restarts, and network interruption can be demonstrated. They do not protect against failure of the physical host. The production walkthrough first creates a portfolio deployment on one Ubuntu server because it is reproducible and affordable. A later subsection explains how to move the same node configuration to three virtual machines.

No Redis command, Redis rate limiting module, third party Raft package, managed consensus service, or external database is used. The only application dependencies are bbolt and the Prometheus Go client. Standard library packages handle HTTP, JSON, templates, cryptography, logging, synchronization, and testing.

0.4 Definition of complete

The project is complete only when all of these statements are true:

- A three node cluster elects exactly one leader under normal conditions.
- A policy and API client created through the admin interface appear on every node after commit.

- Concurrent requests for one subject never produce more allowed cost than the committed token bucket state permits.
- A request is answered only after its decision command is committed and applied.
- Repeating the same idempotency key returns the original decision without charging twice.
- Stopping the leader causes a new leader to appear and decisions resume within the documented timeout.
- Restarted nodes recover their term, vote, log, and applied rate limiting state from disk.
- Loss of quorum makes mutation endpoints return a clear service unavailable response instead of accepting unsafe writes.
- Admin and client APIs enforce authentication, validation, request size limits, secure headers, and safe secret handling.
- Unit, integration, race, security, failover, and load tests pass.
- Docker Compose starts the complete cluster, gateway, and Prometheus with persistent data.
- The README contains architecture, setup, tests, screenshots, limitations, and a short demonstration script.

1. Project Overview

1.1 What the project is

QuorumLimiter is a distributed HTTP service that decides whether a caller may perform an action. A caller might ask, for example, whether user `customer_4821` may send another password reset email under a policy that allows five requests per minute. The service returns an allowed or denied decision plus the remaining token balance and the time at which another token should become available.

A normal in memory rate limiter works only inside one process. If traffic is spread across three independent servers, each server could believe the user still has five requests available. The user might then receive fifteen accepted requests. QuorumLimiter avoids this by placing every state changing decision in a replicated log. A majority of nodes must store the command before the leader applies and returns its result.

1.2 The problem it solves

Rate limits protect systems from accidental overload, abusive clients, expensive repeated work, and unfair resource use. Common examples include login attempts, one time password requests, document exports, payment retries, AI inference calls, and public API traffic.

The harder problem is consistency across nodes. QuorumLimiter solves the educational version of that problem by combining:

- A token bucket state machine for limit decisions.
- A leader that orders every mutation.
- A Raft lite replicated log that survives one node failure.
- Idempotency records that make client retries safe.
- Durable local storage on all three nodes.

1.3 Intended users

The running product has three user groups.

- Application developers call the decision API before performing a protected action.
- Service administrators create policies, issue or revoke API keys, view cluster health, and inspect recent decisions.
- Engineers operate the cluster, monitor metrics, test failover, back up data, and diagnose replication problems.

The repository is also a portfolio artifact. A reviewer should be able to start it, read the architecture decision records, stop a leader, watch a new election, run a load test, and inspect tests that defend important safety properties.

1.4 Main features

- Fixed three node cluster with automatic leader election.
- Token bucket policies with capacity, refill amount, refill interval, maximum cost, and active status.
- Strongly ordered decision mutations committed by quorum.
- Deterministic integer arithmetic rather than floating point token calculations.
- Public decision API authenticated by generated API keys.
- Idempotency key support for safe retries.
- Admin login with signed secure sessions and cross site request forgery protection.
- Policy management, API client creation and revocation, recent decision audit, and cluster status pages.
- Local bbolt storage for persistent Raft metadata, log entries, and applied state.
- Structured JSON logs, health endpoints, readiness checks, and Prometheus metrics.
- Docker Compose environment with isolated node volumes and a Caddy gateway.
- Unit tests, multi node integration tests, race tests, process restart tests, fault scripts, and k6 load tests.
- CI workflow that runs formatting, tests, race detection, linting, and vulnerability checks without using private credentials.

1.5 Skills demonstrated

Building this project may demonstrate more than familiarity with an API framework. It shows state machine design, concurrent programming, network protocols, persistence, failure handling, deterministic computation, authentication, observability, testing under faults, containerization, and technical documentation.

The strongest interview discussion is not simply that the project uses Raft. It is why a decision is returned only after quorum commit, why floating point arithmetic is avoided, how the current term rule protects commit safety, what happens during a partition, and where this educational implementation stops.

1.6 Terms used throughout the guide

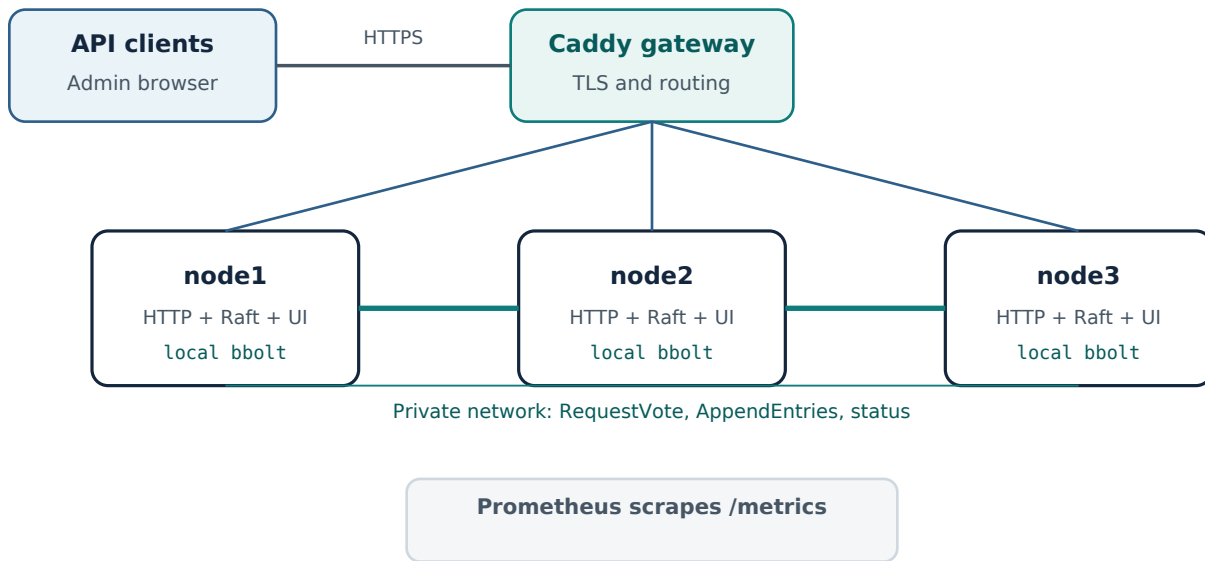
Term	Beginner friendly definition
Token bucket	A counter that holds tokens up to a capacity. Requests spend tokens, while time adds tokens back at a configured rate.
Subject	The identity being limited, such as a user ID, IP address, tenant, or endpoint key.
State machine	Code that produces the same next state whenever it receives the same command in the same order.
Consensus	Agreement among nodes about the same ordered sequence of committed commands.
Term	A logical election number. New elections use larger terms, which helps nodes reject stale leaders.
Quorum	A majority of nodes. In a three node cluster, a quorum is two nodes.
Replicated log	An ordered list of commands stored on multiple nodes.
Commit index	The highest log position known to be safely committed.
Idempotency	The property that retrying the same operation returns the same result without repeating its effect.
Linearizable write	A write that appears to occur once at a single point between request and response.
Network partition	A condition where some nodes cannot communicate even though they may still be running.

2. Final Product Layout

2.1 How a user reaches the product

The public gateway listens on `http://localhost:8080` during development. In production it listens on a domain such as `https://limiter.example.com`. Caddy sends incoming requests to available QuorumLimiter nodes. If a write reaches a follower, that node forwards the request to the known leader. If no leader is known, it returns `503 Service Unavailable with Retry-After: 1`.

Application traffic uses JSON under `/v1`. Administrators use the browser pages under `/admin`. Prometheus connects privately to `/metrics`. Raft calls use `/internal/raft` and must never be routed from the public internet.



2.2 Screen inventory

Screen	Route	Main content	Main actions
Admin sign in	/admin/login	Product name, admin token field, sign in button, generic error message	Sign in
Overview	/admin	Leader, term, quorum, request totals, node table, quick decision test, recent warnings	Run test, open cluster details
Policies	/admin/policies	Search, status filter, policy table, create form	Create, edit, activate, deactivate
API clients	/admin/clients	Client table, permissions, status, creation form	Create client, copy new key once, revoke
Decisions	/admin/decisions	Recent allowed and denied decisions with filters	Filter, inspect details, copy request ID
Cluster	/admin/cluster	Node role, term, commit index, applied index, log size, last heartbeat	Refresh, copy diagnostic summary
API examples	/admin/docs	Curl examples, response meanings, authentication notes	Copy example

2.3 Screen details

2.3.1 Admin sign in

The sign in form has one password style field labeled `Admin token` and one `Sign in` button. It never displays whether the token length or value was close to correct. The server compares the submitted token with `QL_ADMIN_TOKEN` using constant time comparison. Successful login creates an eight hour, signed, HTTP only session cookie. Production cookies also have the `Secure` attribute.

Five failed attempts from the same source IP within one minute should produce 429 Too Many Requests. The page does not include client side analytics or external scripts.

2.3.2 Overview dashboard

The top bar shows the product name, the node serving the page, its role, and current term. A left menu links to all admin pages. Summary cards show leader ID, current term, healthy node count, allowed requests in the last minute, and denied requests in the last minute.

The node table lists node ID, role, health, commit index, applied index, and last contact. The quick test form asks for policy, subject, and cost. Its Run decision button submits an authenticated admin test call and shows the exact response. This is useful for demos, but the form must state that it changes bucket state.

QuorumLimiter node2 | follower | term 18

Overview

- Policies
- API clients
- Decisions
- Cluster
- Docs
- Sign out

Cluster overview

LEADER
node1

TERM
18

QUORUM
3 / 3

Nodes			
node1	leader	healthy	42
node2	follower	healthy	42
node3	follower	healthy	42

Quick decision test

Last minute
1,204 allowed | 38 denied

2.3.3 Policies page

The page begins with a Create policy button. Pressing it expands a form with:

- Name, between 3 and 64 visible characters.
- Policy ID, lowercase letters, numbers, and underscores, between 3 and 48 characters.
- Capacity, an integer from 1 through 1,000,000 tokens.
- Refill tokens, an integer from 1 through 1,000,000.
- Refill interval, an integer plus a unit selector for seconds, minutes, or hours.
- Maximum request cost, an integer from 1 through capacity.
- Active checkbox.
- Save policy and Cancel buttons.

The policy table shows name, ID, capacity, readable refill rate, maximum cost, status, version, and last update. Edit opens the same fields. Deactivate requires a confirmation dialog and does not delete

historical bucket or audit records. Edits are state changing Raft commands and only display success after commit.

2.3.4 API clients page

The creation form asks for a client name and allowed policies. `All policies` is explicit rather than implied by an empty selection. After commit, the server displays a generated key once in a highlighted panel with `Copy key` and `I saved it` buttons. The raw key is not stored and cannot be recovered.

The client table shows name, client ID, key prefix, policy access, status, creation time, and revocation time. Revoke requires confirmation. There is no key edit action. Rotation means creating a new key, updating the application, confirming traffic, and then revoking the old key.

2.3.5 Decisions page

The page shows the latest 100 audit records by default. Filters include result, policy, client, subject, and request ID. Each row includes timestamp, request ID, client, policy, redacted subject, cost, allowed status, remaining whole and fractional tokens, leader term, and log index.

Subjects may contain user identifiers. The default table shows the first four and last four characters with the middle redacted. A local development flag can reveal full values, but production should keep redaction enabled.

2.3.6 Cluster page

The cluster page is read only. It displays each node's advertised address, role, term, leader ID, last log index, commit index, last applied index, database size, and time since leader contact. A colored badge marks healthy, catching up, unavailable, or no quorum states.

There is deliberately no `Force leader`, `Delete log`, or `Reset node` button. Those controls would make a beginner friendly dashboard dangerously destructive. A `Copy diagnostic summary` button copies nonsecret state for issue reports.

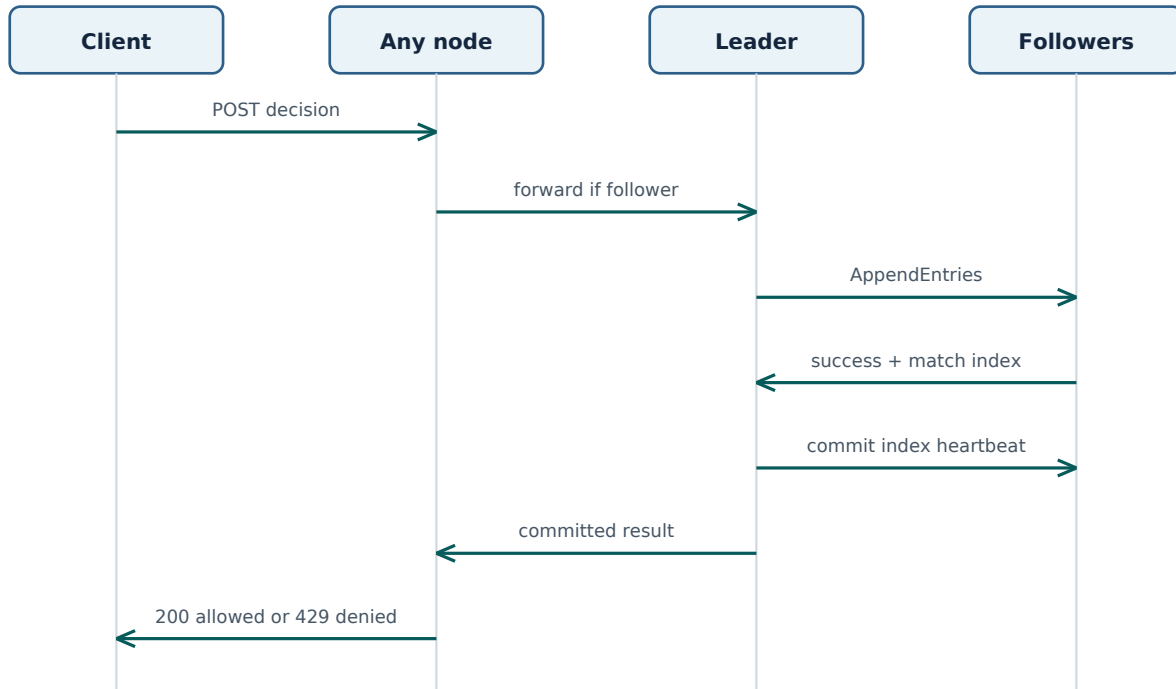
2.3.7 API examples page

This page provides a decision curl command with placeholder values, explains 200, 401, 404, 409, 422, 429, and 503 responses, and reminds the user to supply a fresh idempotency key for a new logical request. It never inserts a real API key into page source.

2.4 Important user flows

Client decision flow

1. The application generates a UUID or another unique request ID.
2. It sends policy ID, subject, and cost with an API key and `Idempotency-Key`.
3. Any node validates the request or forwards it to the leader.
4. The leader appends a decision command to its log.
5. At least one follower stores the entry, creating a two node quorum.
6. The leader advances commit index and applies commands in order.
7. Every applying node runs the same integer token bucket calculation.
8. The leader returns 200 for allowed or 429 for denied.
9. A retry with the same idempotency key returns the stored result.



Administrator setup flow

1. The administrator signs in using the token held in their password manager.
2. They create a policy such as five reset emails per minute with capacity five.
3. They create an API client allowed to use that policy.
4. They copy the raw client key once and place it in the calling application's secret manager.
5. They run the quick test and inspect the decision record.
6. They confirm all nodes have matching commit and applied indexes.

Leader failure flow

1. An operator notes the current leader on the Cluster page.
2. They stop only that container.
3. Followers stop receiving heartbeats and begin an election after randomized timeouts.
4. One follower wins a majority and becomes leader.
5. Requests briefly receive retries or 503, then resume through the new leader.
6. The old node restarts as a follower and catches up from the leader.

3. Technical Architecture

3.1 Version locked technology stack

These versions were checked against official release pages on July 29, 2026. Patch updates may become available later. Finish the first working build with these versions before upgrading one dependency at a time.

Layer	Technology and version	Why it is used
Language	Go 1.26.5	Concurrency, static types, fast compilation, standard HTTP and cryptography packages
Embedded storage	go.etcd.io/bbolt v1.5.0	Durable local transactions without an external database
Metrics library	github.com/prometheus/client_golang v1.24.1	Standard Prometheus instrumentation for Go
Web UI	Go html/template, HTML5, CSS, vanilla JavaScript	Small attack surface and no separate Node.js build
Monitoring server	Prometheus 3.13.1 LTS	Scrapes and queries node metrics
Containers	Docker Engine 29.6.1	Reproducible node processes and volumes
Orchestration	Docker Compose 5.3.0	One file starts three nodes, gateway, and monitoring
Gateway	Caddy 2.11.4	Reverse proxy and automatic production HTTPS
Load testing	k6 2.1.0	Repeatable concurrency and latency tests
Linting	golangci-lint 2.12.2	Static checks beyond go vet
Vulnerability scan	govulncheck 1.1.4	Reachability aware Go vulnerability scanning

3.2 Front end architecture

The dashboard is rendered by Go's `html/template` package. Templates are embedded into the binary with `//go:embed`, which prevents missing template files in a container. A shared base template supplies the top bar, navigation, flash messages, content security policy nonce, and footer.

CSS uses custom properties for color, spacing, typography, focus outlines, and status badges. Layout uses CSS Grid and Flexbox with breakpoints at 900 and 640 pixels. JavaScript is limited to confirmation dialogs, clipboard copying, periodic status refresh, form disclosure, and progressive enhancement. Every important action must work without JavaScript.

The browser never calls internal Raft endpoints. Admin forms post to same origin handlers with a session cookie and cross site request forgery token. JSON admin endpoints exist for testability and are protected by the same session and origin checks.

3.3 Back end architecture

One Go binary runs all application roles. Each process creates:

- A configuration loader that validates all environment values before listening.
- A bbolt store with schema migration and separate read and write transactions.
- A Raft node with one serialized event loop.
- An HTTP server with public, admin, health, metrics, and internal route groups.
- A deterministic token bucket state machine.

- An apply result registry that wakes the waiting HTTP request after its log entry is applied.
- Structured logging with `log/slog`.
- Prometheus collectors.

The Raft event loop owns volatile consensus state. RPC handlers decode and validate messages, then pass them through channels to the event loop. This reduces shared memory races. Disk operations use short bbolt transactions and never hold the Raft mutex during network calls.

3.4 Raft state and rules

Persistent Raft state includes current term, voted for, and all log entries. Volatile state includes commit index and last applied, although this guide also checkpoints them for faster restart. Leaders additionally track next index and match index for each follower.

Election timeouts are randomized independently between 800 and 1400 milliseconds. Heartbeats run every 200 milliseconds. A candidate increments its term, votes for itself, persists both changes before sending requests, and becomes leader only after receiving two votes in the same term.

`AppendEntries` succeeds only when the follower contains the requested previous log index and term. A conflict causes the leader to move that follower's next index backward and retry. A leader advances commit index to an index `N` only when a majority has `matchIndex >= N` and the entry at `N` belongs to the leader's current term. This current term rule is easy to miss and is important for Raft safety.

All committed commands are applied in ascending log order. Application code must never apply an uncommitted entry. A node that sees a higher term immediately persists the term, clears its vote, and steps down.

3.5 Deterministic token bucket

Floating point results can vary subtly across implementations and are inappropriate in a replicated state machine. QuorumLimiter stores token balances in milli tokens, where 1000 milli tokens equal one token.

A policy has whole token capacity, whole refill tokens, refill interval in milliseconds, and maximum whole token cost. A bucket stores:

- `tokens_milli`, from zero through `capacity * 1000`.
- `last_refill_ms`, a Unix time in milliseconds supplied by the leader command.
- `refill_remainder`, the numerator remainder from integer division.

When applying a decision, use `effective_time = max(command.observed_at_ms, bucket.last_refill_ms)`. This prevents time from moving backward after a leader change or clock correction. Refill uses:

```
numerator = elapsed_ms * refill_tokens * 1000 + refill_remainder
added_milli = numerator / refill_interval_ms
new_remainder = numerator % refill_interval_ms
tokens_milli = min(capacity * 1000, tokens_milli + added_milli)
```

If the bucket reaches capacity, reset the remainder to zero because older fractional credit must not let it exceed capacity. Validate numeric bounds and use checked multiplication before computing the numerator.

The command contains the leader observed timestamp. Followers do not call their local clocks while applying the command. The idempotency lookup happens before refill or deduction. An existing record returns the original result unchanged.

When a policy is updated, the update command deterministically visits every existing bucket with that policy prefix. It refills each bucket under the old policy up to the update timestamp, clamps it to the new capacity, resets a remainder whose denominator has changed, records the new policy version, and then stores the policy. This can make a large policy update expensive, but it prevents a new refill rate from being applied retroactively to old elapsed time. Versioned policy history and lazy conversion are future optimizations.

3.6 Database choice and write ownership

bbolt stores key value pairs in named buckets inside one file. Each node has its own file. Raft log replication keeps logical application state aligned, not shared file access.

Only two components may write:

- The Raft persistence layer writes term, vote, log entries, commit checkpoint, and applied checkpoint.
- The deterministic state machine writes policies, clients, rate buckets, idempotency records, and audits while applying committed commands.

HTTP handlers never directly write application buckets. They propose commands and wait for committed apply results. This is a critical architecture rule and should be enforced in code review.

Application writes for one log entry and the new `last_applied` value must commit in the same bbolt transaction. If a process crashes, it must see either neither change or both changes. A separate applied checkpoint transaction could repeat a nonidempotent admin command after restart.

3.7 APIs and external services

The service exposes JSON, HTML, health, metrics, and internal consensus endpoints documented in Section 8. It does not depend on a paid API, cloud database, Redis, email service, or identity provider. The only external network calls in normal operation are calls among cluster nodes and Prometheus scrapes.

This absence is useful. A reviewer can disconnect the internet after pulling images and still run the demonstration.

3.8 Authentication and security

Client keys have the form `qlk_<8 character prefix>_<32 random bytes encoded with base64url>`. The server displays the complete value once. It stores a key prefix for lookup and an HMAC SHA 256 digest made with `QL_API_KEY_PEPPER`. Authentication recomputes the HMAC and compares it in constant time.

The admin token comes from `QL_ADMIN_TOKEN`. The server never logs it. A successful login creates a signed session containing issue time, expiry, and a random cross site request forgery value. The signature uses `QL_SESSION_KEY`. All nodes share the same admin and session secrets so a browser can move between nodes.

Internal Raft requests carry `Authorization: Bearer <QL_CLUSTER_TOKEN>` and travel only on a private network. Production firewalls block internal routes from the public interface. This is reasonable for the portfolio version. Mutual TLS between nodes is a future improvement.

Additional controls include:

- Maximum JSON body size of 64 KiB.
- Read header timeout, read timeout, write timeout, idle timeout, and graceful shutdown.
- Strict JSON decoding with unknown fields rejected.
- Content type checks.

- Input length and character validation.
- Constant time secret comparisons.
- HTTP only, Secure, SameSite Strict session cookies in production.
- Content Security Policy, frame denial, no sniff, and referrer policy headers.
- Login attempt throttling.
- Redacted structured logs.
- No secrets in command line arguments, Git, HTML, metrics labels, or error responses.

3.9 Hosting and deployment

The reproducible deployment uses Docker Compose on Ubuntu. Three named volumes store three independent databases. Caddy is the only public service. Prometheus is bound to loopback or protected by a firewall. Containers run as nonroot users, use read only root filesystems where possible, drop Linux capabilities, and define health checks.

For stronger infrastructure fault tolerance, run one node per virtual machine on a private network and keep fixed peer addresses. Place Caddy or a managed TCP and HTTP load balancer in front. Because membership is fixed, replacing a machine must preserve its node ID and restored data or follow the documented fresh follower procedure.

3.10 Testing architecture

Pure unit tests cover token calculations, validation, authentication helpers, log matching, vote rules, and commit selection. An in process test cluster uses real HTTP listeners on loopback and temporary bbolt files. A deterministic transport or fault proxy can drop, delay, and duplicate messages.

The most important tests assert properties rather than internal method calls:

- Never more than one leader in one term.
- A committed log entry is not replaced.
- Applied indexes are monotonic.
- All healthy nodes converge to the same committed application state.
- Duplicate request IDs do not deduct twice.
- No quorum means no successful mutation.
- A restarted node recovers and catches up.

Race tests use `go test -race ./...`. Load tests use the pinned k6 container. Security checks use `go vet`, `golangci-lint`, `govulncheck`, secret scans in CI, and manual header and authorization tests.

3.11 Known limitations

The API has a single write leader, so maximum write throughput is limited by one node, disk flushes, and quorum latency. Every decision creates a log entry and audit record. Without compaction, disk usage grows. The service depends on reasonably synchronized clocks for refill fairness, although deterministic timestamps and monotonic effective time prevent nodes from disagreeing.

The project prioritizes correctness and clarity over maximum speed. Performance claims should always include the test hardware, policy distribution, concurrency, durability settings, and whether a leader failure occurred.

4. Prerequisites and Setup

4.1 Required hardware, software, and accounts

You need:

- A computer with Ubuntu 24.04 LTS, or Windows 11 with WSL2 Ubuntu 24.04.
- At least 8 GB memory, four CPU cores recommended, and 15 GB free storage.
- Internet access for initial downloads.
- A code editor such as Visual Studio Code.
- An AI coding agent that can read and edit the repository.
- A GitHub account only when you are ready to publish. No credential is needed during local development.
- A domain and Ubuntu server only for the public deployment phase.

You do not need Redis, PostgreSQL, Node.js, Kubernetes, a cloud API key, or a paid monitoring account.

4.2 Knowledge expected

You should be comfortable opening a terminal, changing directories, editing a text file, and reading a basic HTTP request. Go experience is helpful but not required. Before the Raft phases, read Sections 1, 2, 3, and the linked Raft paper once. Do not ask the coding agent to build consensus code that you cannot yet explain at a high level.

4.3 Install base tools on Ubuntu

Run this in an Ubuntu terminal:

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl jq git make openssl
git --version
make --version
```

Expected output includes Git 2.43 or newer and GNU Make. If `apt-get update` fails, fix the system clock and package repository before continuing.

4.4 Install Go 1.26.5

The following commands download the official archive, obtain its published SHA 256 value from the official Go JSON feed, verify the file, and install it. Run them in an Ubuntu terminal. On an ARM64 computer, replace `amd64` with `arm64`.

```
cd /tmp
GO_FILE="go1.26.5.linux-amd64.tar.gz"
curl -fsSL0 "https://go.dev/dl/${GO_FILE}"
GO_SHA="{
  curl -fsSL "https://go.dev/dl/?mode=json&include=all" |
  jq -r --arg file "${GO_FILE}" \
    '.[ ] | .files[] | select(.filename == $file) | .sha256'
}"
test -n "$GO_SHA"
echo "${GO_SHA} ${GO_FILE}" | sha256sum -c -
sudo rm -rf /usr/local/go
sudo tar -C /usr/local -xzf "${GO_FILE}"
echo 'export PATH="/usr/local/go/bin:${go env GOPATH}/bin:$PATH"' >> "$HOME/.profile"
export PATH="/usr/local/go/bin:${go env GOPATH}/bin:$PATH"
go version
```

Expected output is `go version go1.26.5 linux/amd64`. If the checksum command says `FAILED`, delete the archive and download it again. Do not install a file that failed verification.

4.5 Install Docker Engine and Compose

Run these commands in Ubuntu. They use Docker's official apt repository.

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \
  https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" |
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt-get update
sudo apt-get install -y \
  docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin

sudo systemctl enable --now docker
sudo usermod -aG docker "$USER"
newgrp docker
docker version
docker compose version
docker run --rm hello-world
```

The recommended versions are Docker Engine 29.6.1 and Compose 5.3.0. A newer patch release is acceptable if the exact package is no longer offered. If Docker reports permission denied after `usermod`, sign out and sign back in.

4.6 Install Go quality tools

Run these commands from any directory after Go is on PATH:

```
go install \
  github.com/golangci/golangci-lint/v2/cmd/golangci-lint@v2.12.2
go install \
  golang.org/x/vuln/cmd/govulncheck@v1.1.4
golangci-lint version
govulncheck -version
```

If the commands install successfully but cannot be found, run:

```
export PATH="/usr/local/go/bin:$(go env GOPATH)/bin:$PATH"
```

4.7 Create the repository

Choose a workspace directory and run:

```
mkdir -p "$HOME/projects"
cd "$HOME/projects"
mkdir quorumlimiter
cd quorumlimiter
git init
go mod init github.com/yourname/quorumlimiter
go mod edit -go=1.26.0
go mod edit -toolchain=go1.26.5
go get go.etcd.io/bbolt@v1.5.0
go get github.com/prometheus/client_golang@v1.24.1
go mod tidy
```

Replace `yourname` before running `go mod init`. Use your intended GitHub username or organization, but do not create or push a remote yet. Expected files are `go.mod` and `go.sum`.

4.8 Create safe development secrets

Secrets belong in an ignored `.env` file. The repository will contain `.env.example` with placeholders. Run from the repository root:

```
umask 077
QL_CLUSTER_TOKEN="$(openssl rand -base64 36 | tr -d '\n')"
QL_ADMIN_TOKEN="$(openssl rand -base64 36 | tr -d '\n')"
QL_SESSION_KEY="$(openssl rand -base64 48 | tr -d '\n')"
QL_API_KEY_PEPPER="$(openssl rand -base64 48 | tr -d '\n')"

printf '%s\n' \
  "QL_CLUSTER_TOKEN=${QL_CLUSTER_TOKEN}" \
  "QL_ADMIN_TOKEN=${QL_ADMIN_TOKEN}" \
  "QL_SESSION_KEY=${QL_SESSION_KEY}" \
  "QL_API_KEY_PEPPER=${QL_API_KEY_PEPPER}" \
  > .env
chmod 600 .env
unset QL_CLUSTER_TOKEN QL_ADMIN_TOKEN QL_SESSION_KEY QL_API_KEY_PEPPER
```

Add `.env`, `data/`, `coverage/`, and local binaries to `.gitignore` before the first commit. Never paste the generated values into an AI conversation, issue, screenshot, test fixture, or commit.

4.9 Initial verification

Run from the repository root:

```
go version
go env GOMOD
go list -m all
docker version --format '{{.Server.Version}}'
docker compose version
git status --short
```

You should see Go 1.26.5, the current repository's `go.mod`, `bbolt 1.5.0`, `Prometheus client 1.24.1`, a running Docker server, and only the new project files.

Pause before implementation

Give the coding agent one phase prompt at a time. Review its changed files and test output before moving forward. If a phase fails, fix it in that phase instead of asking the agent to continue building on an uncertain base.

5. Project Folder Structure

5.1 Complete recommended tree

```
quorumlimiter/  
  .github/  
    workflows/  
      ci.yml  
  cmd/  
    quorumlimiter/  
      main.go  
  deploy/  
    Caddyfile  
    compose.dev.yml  
    compose.prod.yml  
    prometheus.yml  
  docs/  
    api.md  
    architecture.md  
    demo.md  
    security.md  
  adr/  
    0001-fixed-three-node-raft.md  
    0002-deterministic-token-bucket.md  
    0003-bbolt-storage.md  
  internal/  
    api/  
      admin.go  
      admin_test.go  
      decision.go  
      decision_test.go  
      errors.go  
      internal_raft.go  
      middleware.go  
      routes.go  
      types.go  
    auth/  
      apikey.go  
      apikey_test.go  
      csrf.go  
      session.go  
      session_test.go  
    config/  
      config.go  
      config_test.go  
    dashboard/  
      handlers.go  
      viewmodels.go  
      templates/  
        base.html  
        login.html  
        overview.html
```

```
    policies.html
    clients.html
    decisions.html
    cluster.html
    docs.html
  static/
    app.css
    app.js
  limiter/
    command.go
    models.go
    state_machine.go
    token_bucket.go
    token_bucket_test.go
  metrics/
    metrics.go
  raft/
    apply.go
    election.go
    election_test.go
    node.go
    node_test.go
    replication.go
    rpc.go
    types.go
    waiters.go
  server/
    server.go
    shutdown.go
  storage/
    encoding.go
    migrations.go
    raft_store.go
    raft_store_test.go
    state_store.go
    store.go
    testhelpers_test.go
  testcluster/
    cluster.go
    faults.go
    wait.go
  pkg/
    client/
      client.go
      client_test.go
  scripts/
    failover.sh
    load.js
```

```
  smoke.sh
  test/
    integration/
      cluster_test.go
      idempotency_test.go
      partition_test.go
      recovery_test.go
  .dockerignore
  .env.example
  .gitignore
  .golangci.yml
  Dockerfile
  LICENSE
  Makefile
  README.md
  go.mod
  go.sum
```

5.2 Folder responsibilities

Path	Purpose
cmd/quorumlimiter	Small executable entry point. It loads config, opens storage, creates services, starts servers, and handles shutdown.
internal/config	Environment parsing, defaults, duration handling, peer parsing, and startup validation.
internal/storage	bbolt ownership, bucket migrations, encoding helpers, Raft persistence, and application state reads and writes.
internal/raft	Consensus states, event loop, elections, replication, commit advancement, application, and RPC messages.
internal/limiter	Policy and bucket models, replicated command types, deterministic state machine, and token math.
internal/auth	Client key hashing, admin session signing, cross site request forgery protection, and constant time comparisons.
internal/api	HTTP routes, request and response data transfer objects, validation, middleware, and Raft RPC handlers.
internal/dashboard	Server rendered pages, templates, styles, JavaScript, and view models.
internal/metrics	Prometheus metric definitions and safe label choices.
internal/server	HTTP server timeouts, startup, graceful shutdown, and shared dependencies.
internal/testcluster	Reusable test nodes, temporary stores, fake clocks, wait helpers, and fault controls.
pkg/client	A small public Go client for the decision API. It must not import internal packages.
test/integration	Multi node behavior, failure, partition, retry, restart, and convergence tests.
deploy	Local and production Compose definitions, Caddy configuration, and Prometheus scrape configuration.
scripts	Human runnable smoke, failover, and k6 load scenarios.
docs/adr	Architecture decision records that explain important tradeoffs and limitations.

5.3 Important file rules

`main.go` should remain small. Business logic belongs in testable packages. `internal/raft` must not import HTTP handlers or dashboard code. `internal/limiter` must not read a clock, environment variable, or network connection while applying commands. `internal/storage` owns bbolt bucket names and encoding.

Templates are presentation only. They receive prepared view models and do not perform storage access. The public package under `pkg/client` may be imported by other Go projects, so keep its API stable and document it.

The repository root contains operator entry points. `Makefile` supplies short, documented commands. `.env.example` contains names and safe placeholders only. The real `.env` stays ignored. `README.md` tells a reviewer how to run the project in less than ten minutes.

6. Complete Development Plan

The phases below are intentionally sequential. Each phase ends with a test gate. Do not let an AI coding agent start the next phase until the stated commands pass and you understand the changed files.

6.1 Phase 1: Bootstrap, configuration, and process lifecycle

Objective

Create a compiling Go service with validated configuration, structured logs, health endpoints, graceful shutdown, repository guardrails, and no consensus behavior yet.

Why this phase is necessary

Distributed code becomes difficult to debug when startup behavior is vague. A strict configuration layer makes every node's identity, peer set, timeout, storage path, and secrets explicit before concurrency begins.

Exact actions

1. Create the folder tree from Section 5.
2. Add `.gitignore`, `.dockerignore`, `.env.example`, `Makefile`, and the three initial architecture decision records.
3. Define a `Config` struct with node ID, bind address, advertised URL, peers, data path, timeouts, secrets, public base URL, production flag, and log level.
4. Parse `QL_PEERS` as comma separated `nodeID=URL` pairs. Require exactly three unique nodes including the local node.
5. Require election minimum to be greater than three heartbeat intervals and election maximum to be greater than minimum.
6. Decode session and pepper values from environment without printing them.
7. Create `main.go`, structured logging, signal handling, and a server with `/health/live`.
8. Add table driven configuration tests for valid and invalid cases.

Commands

Run from the repository root:

```
mkdir -p \
  cmd/quorumlimiter \
  deploy docs/adr scripts test/integration \
  internal/{api,auth,config,dashboard,limiter,metrics,raft,server,storage,testcluster} \
  pkg/client

go fmt ./...
go test ./...
go vet ./...
go run ./cmd/quorumlimiter
```

The final command should fail fast with a readable list of missing required variables. Then run one temporary node configuration:

```

set -a
. ./env
set +a
export QL_NODE_ID=node1
export QL_BIND_ADDR=127.0.0.1:18081
export QL_ADVERTISE_URL=http://127.0.0.1:18081
export QL_PEERS='node1=http://127.0.0.1:18081,node2=http://127.0.0.1:18082,node3=http://127.0.0.1:18083'
export QL_DATA_PATH=./data/node1/quorumlimiter.db
export QL_ELECTION_TIMEOUT_MIN=800ms
export QL_ELECTION_TIMEOUT_MAX=1400ms
export QL_HEARTBEAT_INTERVAL=200ms
export QL_PROPOSAL_TIMEOUT=3s
export QL_PUBLIC_BASE_URL=http://localhost:8080
export QL_PRODUCTION=false
go run ./cmd/quorumlimiter

```

In a second terminal:

```
curl -i http://127.0.0.1:18081/health/live
```

Files to create or modify

Create `cmd/quorumlimiter/main.go`, `internal/config/config.go`, `internal/config/config_test.go`, `internal/server/server.go`, `internal/server/shutdown.go`, root guardrail files, and the first three decision records. Do not create Raft implementation files beyond empty package documentation.

What the AI coding agent should produce

The agent should produce compiling code, focused configuration tests, a Makefile with `fmt`, `test`, `race`, `lint`, `vuln`, and `run` targets, and concise documentation. It should report every file changed and the exact test commands run.

How to test

Confirm missing secrets cause startup failure, invalid peer entries are rejected, duplicate node IDs fail, invalid durations fail, and valid values start the server. Send `SIGTERM` and confirm the process exits cleanly within five seconds.

Expected output

`GET /health/live` returns `200` and JSON similar to:

```
{"status":"alive","node_id":"node1"}
```

Logs are JSON and contain time, level, message, and node ID. They contain no secret values.

Common errors and corrections

- If brace expansion creates unexpected directories, run the `mkdir` command in Bash rather than another shell.
- If `.env` is printed by `git status`, verify `.gitignore` contains a line with exactly `.env`.
- If the server hangs on shutdown, ensure `http.Server.Shutdown` receives a bounded context and all background goroutines observe cancellation.
- If configuration tests modify global environment in parallel, use `t.Setenv` and do not mark those tests parallel.

6.2 Phase 2: bbolt storage and migrations

Objective

Create a durable store with explicit bucket migrations, typed Raft persistence methods, application state access, safe encoding, and temporary database tests.

Why this phase is necessary

Consensus safety depends on writing the current term and vote before responding. Restart recovery depends on exact log ordering. Storage behavior should be verified before elections and network calls are added.

Exact actions

1. Add a `Store` type that opens one bbolt file with mode `0600` and a one second open timeout.
2. Create schema version 1 and the buckets listed in Section 8.
3. Store unsigned integers as eight byte big endian values so log keys sort numerically.
4. Define methods for term, vote, log append, log truncation, entry lookup, first and last index, commit checkpoint, and applied checkpoint.
5. Define read methods for policies, clients, recent audits, node metadata, and decision idempotency records.
6. Keep application mutation methods unexported or require a state machine transaction callback.
7. Make migrations idempotent and transactional.
8. Add tests for first open, reopen, log order, truncation, missing keys, transaction rollback, and file permissions.

Commands

```
go test ./internal/storage -count=1
go test -race ./internal/storage -count=1
go test ./...
```

Inspect a disposable test database only through a small debug command or test helper. Do not edit the binary file manually.

Files to create or modify

Create all files under `internal/storage` shown in Section 5. Add storage interfaces only where they improve testability. Update `main.go` so it opens the store and closes it during shutdown.

What the AI coding agent should produce

The agent should implement short bbolt transactions, copy byte slices that must survive a transaction, wrap errors with operation context, and keep bucket names private. It should not place API JSON models directly in storage.

How to test

Write term 7 and vote `node2`, close the store, reopen it, and read the same values. Append entries at indexes 1 through 4, truncate from index 3, and confirm indexes 1 and 2 remain while 3 and 4 are absent. Force a callback error and confirm no partial state was committed.

Expected output

All storage tests pass under the race detector. Reopening a file returns the exact persisted term, vote, log, commit checkpoint, and applied checkpoint.

Common errors and corrections

- `timeout` while opening usually means another process holds the database file. Stop that process or use a different temporary path.
- `invalid argument` can mean the parent directory does not exist. Create it with mode `0700` before opening the file.
- Values that become corrupted after a transaction often reference bbolt managed memory. Copy returned bytes before the transaction ends.
- Lexicographic log ordering is wrong if indexes are stored as decimal strings. Use fixed width big endian keys.

6.3 Phase 3: Deterministic token bucket state machine

Objective

Implement policy, client, bucket, idempotency, and audit commands with deterministic integer arithmetic and exhaustive unit tests.

Why this phase is necessary

Raft only agrees on command order. The nodes still diverge if applying one command can produce different results. This phase proves that the business state machine is deterministic before it is attached to consensus.

Exact actions

1. Define versioned command envelopes with command ID, type, timestamp, and typed payload.
2. Implement commands for create policy, update policy, set policy active status, create client, revoke client, and decide.
3. Validate policy and client fields exactly as documented.
4. Implement milli token refill with remainder preservation, checked arithmetic, monotonic effective time, capacity clamp, and cost deduction.
5. Fill a new subject bucket to full capacity on first use.
6. Look up idempotency before any state change.
7. Store the complete decision result under client ID and request ID.
8. Add a deterministic pruning command for expired idempotency and old audit records. The leader schedules it, while the state machine uses the timestamp inside the command.
9. Make policy updates deterministically refill and convert existing buckets at the update timestamp before storing the new policy.
10. Add table driven tests and a randomized reference model test with a fixed seed.

Commands

```
go test ./internal/limiter -count=1
go test ./internal/limiter -run TestTokenBucket -count=100
go test -race ./internal/limiter -count=1
go test ./...
```

Files to create or modify

Create `internal/limiter/models.go`, `command.go`, `token_bucket.go`, `state_machine.go`, and tests. Add the required state transaction methods in `internal/storage/state_store.go`.

What the AI coding agent should produce

The agent should return pure calculation helpers where possible. It must not use `float32`, `float64`, `time.Now`, random values, goroutines, network calls, or environment variables in command application. It should document arithmetic bounds.

How to test

Include cases for a full new bucket, exact capacity, zero elapsed time, partial refill, remainder carry, refill to capacity, backward timestamp, maximum cost, insufficient tokens, duplicate request, inactive policy, revoked client, unknown policy, and arithmetic overflow rejection.

Also test a refill rate change after several idle seconds. Existing buckets must receive old rate credit only through the policy update timestamp and new rate credit only after that timestamp.

Run the same serialized command sequence against two fresh stores. Export normalized state and compare the bytes. The states must match exactly.

Expected output

Given capacity 5, refill 1 token per 12 seconds, and five immediate cost 1 requests, all five are allowed. The sixth is denied. At 12 seconds, one additional cost 1 request is allowed. Replaying request four with its original idempotency key returns its original allowed result without reducing the current balance.

Common errors and corrections

- Refill drift usually means the integer division remainder was discarded. Store and reuse it.
- Divergence after a leader change usually means followers called their own clocks. Use only the timestamp inside the command.
- A duplicate request that changes `last_refill_ms` is not truly idempotent. Return before all bucket work.
- Integer overflow can silently produce negative values. Validate inputs and use checked multiplication helpers.

6.4 Phase 4: Raft types, persistent transitions, and event loop

Objective

Build the Raft node skeleton, message types, serialized event loop, persistent term and vote transitions, log helpers, and node status reporting.

Why this phase is necessary

Elections and replication share state. A clear owner for that state prevents races and makes term changes auditable. This phase creates the foundation without yet claiming a working election.

Exact actions

1. Define `Role` values follower, candidate, and leader.
2. Define `LogEntry`, `RequestVoteRequest`, `RequestVoteResponse`, `AppendEntriesRequest`, `AppendEntriesResponse`, and `Status`.
3. Give RPC messages protocol version, cluster ID, source node ID, and term.
4. Create a `Node` with context, store, peer clients, event channel, proposal channel, apply channel, and status query channel.
5. Recover term, vote, log bounds, commit checkpoint, and applied checkpoint during construction.
6. Implement `becomeFollower`, `startElectionState`, and `becomeLeaderState` helpers. Higher term transitions must persist before a response.
7. Use one event loop to own role, term, leader ID, commit index, last applied, next indexes, match indexes, and timers.
8. Add a fake clock or injectable timer factory for deterministic tests.
9. Expose immutable status snapshots.

Commands

```
go test ./internal/raft -run 'TestPersistent|TestTransition|TestRecovery' -count=1
go test -race ./internal/raft -count=1
go test ./...
```

Files to create or modify

Create `internal/raft/types.go`, `node.go`, `rpc.go`, and initial tests. Add interfaces for the persistent store, transport, state machine apply function, and timer source.

What the AI coding agent should produce

The agent should keep mutable Raft fields private to the event loop. Network calls must occur outside store transactions. The code should return typed errors for stopped node, not leader, no leader, proposal timeout, and lost quorum.

How to test

Start a node from an empty store and verify follower term zero. Feed a higher term message and verify term persistence plus cleared vote. Restart from the same store and verify recovery. Ask for status concurrently from many goroutines under the race detector.

Expected output

Tests show legal state transitions and no race reports. A node status response includes node ID, role, term, leader ID, last log index, commit index, last applied, and peer progress without exposing secrets.

Common errors and corrections

- A data race means another goroutine reads or writes event loop fields directly. Use query messages or atomic immutable snapshots.
- A node that forgets its vote after restart can vote twice in one term. Persist vote before responding.
- Deadlock during shutdown often comes from sending on an unbuffered channel after the loop exits. Select on context cancellation for every send and wait.

6.5 Phase 5: Leader election and RequestVote

Objective

Implement randomized election timeouts, candidate voting, RequestVote transport, majority election, heartbeat startup, and reliable step down on higher terms.

Why this phase is necessary

The cluster needs exactly one active leader to order writes. Election correctness is also the first visible distributed behavior and a major source of subtle bugs.

Exact actions

1. Reset a follower's election timer only after valid AppendEntries from a current leader or after granting a vote.
2. On timeout, increment term, vote for self, persist both, and send RequestVote concurrently to peers.
3. Grant at most one vote per term and only if the candidate log is at least as up to date.
4. Compare last log term first, then last log index.
5. Count only votes for the current election term.
6. Become leader after self plus one follower vote.
7. Initialize each follower's `nextIndex` to leader last index plus one and `matchIndex` to zero.
8. Append a no operation entry in the new term, then start heartbeats.
9. Step down immediately on any response or request with a higher term.
10. Protect the internal endpoint with the cluster token, request size limit, and source node allowlist.

Commands

```
go test ./internal/raft -run 'TestVote|TestElection|TestHigherTerm' -count=25
go test -race ./internal/raft -run 'TestElection' -count=10
go test ./...
```

Files to create or modify

Modify `internal/raft/election.go`, `node.go`, `rpc.go`, and related tests. Add HTTP transport code in `internal/api/internal_raft.go` only after election behavior passes with a fake transport.

What the AI coding agent should produce

The agent should include deterministic tests for split votes, stale responses, stale candidates, higher term messages, equal logs, more recent term, more recent index, and restart after voting. It should not use arbitrary sleeps in unit tests.

How to test

Use a three node in process cluster. Wait until exactly one node reports leader. Record the term. Stop the leader and confirm one remaining node becomes leader in a higher term. Restart the old leader and confirm it becomes a follower.

Expected output

A healthy cluster elects one leader within roughly 1.5 seconds. Terms are monotonic. The stopped leader is replaced while two nodes remain. No test observes two leaders in the same term.

Common errors and corrections

- Repeated split votes often mean all nodes use the same random seed. Seed each node timer source independently while keeping tests injectable.
- A stale candidate winning usually indicates incorrect log freshness comparison.
- Election storms can happen if every RequestVote request resets the timer. Reset only when granting a valid vote.
- Two leaders in one term usually indicate nonpersistent votes or vote counting across terms. Treat this as a blocking correctness defect.

6.6 Phase 6: Log replication, quorum commit, and ordered apply

Objective

Replicate proposals, repair follower log conflicts, advance commit safely, apply commands in order, return apply results, and recover after restart.

Why this phase is necessary

An elected leader alone does not provide consistency. The service becomes meaningful only when a command is stored on a majority, committed under Raft rules, and applied exactly once in order.

Exact actions

1. Accept proposals only on the leader.
2. Append each proposal to the local durable log before sending it.
3. Send AppendEntries with previous index, previous term, one or more entries, and leader commit index.
4. On a follower, reject inconsistent previous entries. On success, delete conflicting suffix entries and append new entries in one transaction.
5. Return conflict term and first conflict index so the leader can jump backward efficiently.
6. Update next and match indexes only for responses from the current term.
7. Advance commit using majority match indexes and the current term rule.
8. Broadcast the new commit index quickly.
9. Apply indexes from `lastApplied + 1` through `commitIndex` in order.
10. Commit each entry's application state changes and its new applied checkpoint in the same bbolt transaction.
11. Complete the waiting proposal with its deterministic result.
12. Time out the HTTP wait without canceling a command that may later commit. A retry must use the same idempotency key.
13. On startup, replay committed but unapplied entries before becoming ready.

Commands

```
go test ./internal/raft -run 'TestAppend|TestConflict|TestCommit|TestApply' -count=20
go test ./test/integration -run 'TestReplication|TestRecovery' -count=5
go test -race ./internal/raft ./test/integration -count=1
go test ./...
```

Files to create or modify

Create or modify `internal/raft/replication.go`, `apply.go`, `waiters.go`, and their tests. Connect the limiter state machine through a narrow apply interface.

What the AI coding agent should produce

The agent should implement a bounded waiter registry, clean waiter removal, per peer replication workers or event loop tasks, request contexts, and response validation. It must not mark an entry committed only because it exists on the leader. It must atomically commit application state and `last_applied`.

How to test

Propose a policy command and confirm it appears on all healthy nodes. Partition one follower and continue through a two node quorum. Heal it and confirm catch up. Partition the leader away from both followers and confirm its proposals do not succeed. Restart a follower with a shorter and conflicting log and confirm the leader repairs it.

Expected output

Two nodes can commit while one is unavailable. One isolated node cannot commit. Healthy nodes converge to the same log terms, commit index, applied index, policy state, and bucket state.

Common errors and corrections

- A successful response before `lastApplied` reaches the proposal index violates the API contract. Wait for apply, not only append.
- Followers that keep conflicting entries usually truncate after appending. Truncate the conflict suffix first inside the same transaction.
- A dead waiter map indicates timeout paths do not unregister. Add cleanup tests.
- Committing an entry from an older term using only a majority match can violate Raft rules. Require the candidate commit entry to be from the current term.

6.7 Phase 7: Public decision API and leader forwarding

Objective

Expose a secure, documented decision API that works through any node, authenticates clients, validates input, forwards writes to the leader, and maps consensus outcomes to stable HTTP responses.

Why this phase is necessary

The consensus layer needs a usable boundary. Clients should not need to know which node is leader, and retries should be predictable during elections and timeouts.

Exact actions

1. Add `POST /v1/decisions`.
2. Require `Authorization: Bearer <api key>`, `Idempotency-Key`, and `Content-Type: application/json`.
3. Limit the body to 64 KiB and reject unknown JSON fields.
4. Validate policy ID, subject length, subject characters, cost, and request ID length.
5. Authenticate the key using prefix lookup and constant time HMAC comparison.

6. On a follower with a known leader, forward the method, path, safe headers, and body with a two second internal timeout.
7. Add a hop header and reject more than one forward to prevent loops.
8. On unknown leader, return 503 with retry guidance.
9. On the leader, propose a decision command and wait for committed apply.
10. Return stable JSON and headers for allowed, denied, duplicate, not found, unauthorized, invalid, timeout, and unavailable results.
11. Add a small Go client with safe retry only when an idempotency key is present.

Commands

```
go test ./internal/api ./internal/auth ./pkg/client -count=1
go test -race ./internal/api ./pkg/client -count=1
go test ./test/integration -run 'TestDecision|TestForward|TestIdempotency' -count=5
go test ./...
```

After the cluster phase exists, a smoke request will look like:

```
curl -i http://localhost:8080/v1/decisions \
-H 'Authorization: Bearer qlk_EXAMPLE_REPLACE_ME' \
-H 'Idempotency-Key: 01JEXAMPLE0000000000000000' \
-H 'Content-Type: application/json' \
--data '{
  "policy_id": "password_reset",
  "subject": "customer_4821",
  "cost": 1
}'
```

Files to create or modify

Create `internal/api/decision.go`, `types.go`, `errors.go`, `middleware.go`, `routes.go`, `tests`, and `pkg/client/client.go`. Add authentication lookup methods and proposal interfaces.

What the AI coding agent should produce

The agent should produce request and response structs with JSON tags, a consistent error envelope, no secret logging, and tests for every status code. Forwarding must preserve idempotency and trace headers but strip hop by hop headers.

How to test

Send requests through each node address and through Caddy. Confirm all reach the leader and affect one shared logical bucket. Retry the same request ID through a different node. Test invalid keys, revoked keys, inactive policies, missing headers, excessive body, unknown fields, and no leader.

Expected output

An allowed response returns 200, `allowed: true`, remaining milli tokens, reset time, request ID, policy ID, leader term, and log index. A denied response returns 429, `allowed: false`, the same decision fields, and `Retry-After`.

Common errors and corrections

- A forwarding loop means the hop marker is missing or not checked.
- Reauthentication failures on the leader can occur if the follower removes the Authorization header. Forward it only over the private node network.

- A context `deadline` exceeded may still represent a committed command. Tell the client to retry the same idempotency key.
- Metrics with subject or API key labels leak data and create high cardinality. Never use those values as labels.

6.8 Phase 8: Admin authentication and secure mutation APIs

Objective

Implement admin login, signed sessions, cross site request forgery protection, login throttling, secure headers, and Raft backed policy and client administration endpoints.

Why this phase is necessary

The dashboard controls policy and credential state. Building the security boundary before the pages reduces the chance that temporary insecure handlers survive into the final product.

Exact actions

1. Implement GET `/admin/login`, POST `/admin/login`, and POST `/admin/logout`.
2. Compare the submitted token with `QL_ADMIN_TOKEN` in constant time.
3. Create an eight hour signed session cookie with issue time, expiry, and random cross site request forgery token.
4. Verify signature, expiry, and cookie attributes on every admin route.
5. Require a hidden token plus same origin check on state changing form and JSON requests.
6. Rate limit failed sign in attempts by source IP using an in memory bounded map.
7. Add security headers and no store cache headers to admin responses.
8. Implement admin policy create, update, activate, and deactivate endpoints as Raft proposals.
9. Implement client create and revoke. Generate the raw client key on the leader, replicate only prefix and digest, and return the raw value once after commit.
10. Add audit events for admin mutations without storing secrets.

Commands

```
go test ./internal/auth ./internal/api -run 'TestAdmin|TestSession|TestCSRF' -count=10
go test -race ./internal/auth ./internal/api -count=1
go test ./test/integration -run 'TestAdminMutation|TestClientLifecycle' -count=3
go test ./...
```

Files to create or modify

Create `internal/auth/session.go`, `csrf.go`, relevant tests, and `internal/api/admin.go`. Update middleware, route registration, limiter command types, and storage state methods.

What the AI coding agent should produce

The agent should produce a small, readable session format, rotating ready key identification if practical, secure cookie configuration based on `QL_PRODUCTION`, and generic authentication failures. It must never put the raw API key into the Raft log, bbolt, structured log, metric, or audit record.

How to test

Test valid and invalid login, modified cookie, expired cookie, missing cross site request forgery token, wrong origin, repeated failed login, successful logout, policy validation, duplicate policy ID, client creation, key authentication, key revocation, and attempts to reuse the revoked key.

Expected output

Successful sign in sets an HTTP only SameSite Strict cookie. Creating a client returns one raw `qlk_...` value. Reading that client later shows only its ID, name, prefix, permissions, and status.

Common errors and corrections

- Sessions that work on only one node usually use a per process secret. All nodes must share `QL_SESSION_KEY`.
- Cross site request forgery tests can fail behind Caddy if origin reconstruction ignores forwarded protocol. Trust forwarded headers only from the configured gateway network.
- A raw key found in test logs is a release blocker. Redact request headers and response bodies that may contain one time secrets.
- Using a normal equality operator for tokens can leak timing information. Use `crypto/subtle`.

6.9 Phase 9: Admin dashboard and responsive interface

Objective

Build all screens described in Section 2 with accessible server rendered templates, clear feedback, progressive enhancement, and responsive layout.

Why this phase is necessary

A visible interface makes the cluster understandable to a reviewer and gives operators a safer path than hand written database or internal API calls. Server rendering keeps the project achievable for one person.

Exact actions

1. Embed templates and static assets into the binary.
2. Create a base layout with top bar, navigation, flash messages, main content landmark, and sign out form.
3. Build login, overview, policies, clients, decisions, cluster, and API examples pages.
4. Use semantic labels, field descriptions, validation summaries, keyboard focus indicators, and accessible status text.
5. Add forms and buttons exactly as listed in Section 2.
6. Render one time API keys only in the immediate create response. Use no store headers.
7. Add JavaScript only for copy, confirm, form reveal, and optional polling.
8. Add mobile layouts for navigation, cards, forms, and horizontally scrollable tables.
9. Add handler tests and HTML assertions for authentication, cross site request forgery tokens, escaping, and important controls.

Commands

```
go test ./internal/dashboard ./internal/api -run 'TestDashboard|TestPage' -count=1
go test -race ./internal/dashboard ./internal/api -count=1
go test ./...
go run ./cmd/quorumlimiter
```

Use browser developer tools at widths 1440, 1024, 768, 390, and 320 pixels once a local cluster is available.

Files to create or modify

Create all files under `internal/dashboard/templates` and `internal/dashboard/static`, plus handlers and view models. Update route registration and embedded file declarations.

What the AI coding agent should produce

The agent should produce valid templates with no inline event handlers, no external CDN assets, and a restrained interface matching the wireframe. It should include an empty state, loading state where polling is used, error state, and success feedback for every screen.

How to test

Navigate every route while signed out and confirm redirection to login. Sign in and exercise every button. Enter HTML in a permitted text field and confirm it is escaped. Disable JavaScript and confirm forms still work. Use keyboard only and verify visible focus plus logical order.

Expected output

The dashboard provides all screens from Section 2, has no browser console errors, remains usable at 320 pixels, and never displays a stored raw API key.

Common errors and corrections

- `template: pattern matches no files` in a container means embed paths are wrong. Embed relative to the Go source file and test the built binary.
- Missing styles after proxying can mean static routes are behind admin mutation middleware. Serve static files with safe headers but without session requirements.
- Horizontal page overflow often comes from long request IDs. Apply wrapping to identifiers and scroll only the table container.
- Unsafe HTML usually comes from using `template.HTML` on user data. Remove that conversion.

6.10 Phase 10: Health, readiness, logging, and Prometheus metrics

Objective

Make normal operation and failure states observable without exposing secrets or creating high cardinality metrics.

Why this phase is necessary

Distributed failures are often partial. A process can be alive but unable to reach quorum, or a follower can be far behind. Clear readiness, structured logs, and metrics make those differences visible.

Exact actions

1. Keep `/health/live` limited to process liveness.

2. Add `/health/ready`. A leader is ready only if it contacted a quorum recently. A follower is ready only if it knows a leader and is not excessively behind.
3. Add `/metrics` with request counters, duration histogram, decision counters, Raft role, term, elections, RPC counters, log indexes, apply lag, proposal duration, bbolt size, and last quorum contact.
4. Use only bounded labels such as node, route template, method, status class, role, RPC type, and result.
5. Add request IDs to structured logs and redact authorization, cookies, subjects, and one time keys.
6. Log role changes, term changes, vote grants, leader changes, commit advancement, recovery, and shutdown at appropriate levels.
7. Add Prometheus scrape configuration and initial alert expressions in documentation.

Commands

```
go test ./internal/metrics ./internal/api -run 'TestHealth|TestMetrics' -count=1
go test -race ./internal/metrics ./internal/raft -count=1
go test ./...
curl -fsS http://127.0.0.1:18081/metrics | head -n 30
```

Files to create or modify

Create `internal/metrics/metrics.go`, health handlers, logging middleware, `deploy/prometheus.yml`, and observability documentation.

What the AI coding agent should produce

The agent should register metrics once, avoid duplicate collector panics in tests, and use custom registries where helpful. It should state every metric's type, labels, and meaning.

How to test

Assert readiness changes during leader loss and quorum loss. Scrape every node. Search metric text and logs for known test API keys and subjects. Confirm none appear. Confirm label counts remain bounded after sending thousands of unique subjects.

Expected output

Prometheus shows all three nodes as up. A Grafana installation is not required. Prometheus's built in query page can graph `rate(quorumlimiter_decisions_total[1m])` and inspect leader changes.

Common errors and corrections

- Duplicate registration panics occur when package globals register during every test. Accept a registry as a dependency.
- Readiness that always returns 200 hides quorum loss. Base it on recent consensus observations.
- Per subject labels can exhaust memory. Move such detail to redacted audit records, not metrics.
- Logging every heartbeat creates noise and disk use. Count them in metrics and log only state changes or failures.

6.11 Phase 11: Docker image and three node Compose cluster

Objective

Create a reproducible local environment with three node containers, persistent volumes, Caddy routing, Prometheus, health checks, and secure defaults.

Why this phase is necessary

The final project must demonstrate real independent processes, network communication, persistence, restart, and leader replacement. Compose makes the environment repeatable for reviewers.

Exact actions

1. Create a multi stage Dockerfile using `golang:1.26.5-alpine3.23` for build and `alpine:3.23` for runtime.
2. Build with `CGO_ENABLED=0`, `-trimpath`, and version metadata.
3. Create a nonroot runtime user, `/data` ownership, and a health check.
4. Add three node services with fixed IDs, peer URLs, and separate named volumes.
5. Put all nodes, Caddy, and Prometheus on a private bridge network.
6. Publish Caddy at port 8080, node debug ports at loopback only, and Prometheus at loopback only.
7. Configure Caddy to load balance public and admin routes but explicitly reject `/internal/*` and `/metrics`.
8. Add resource limits, restart policy, stop grace period, read only root filesystem, temporary directories, and dropped capabilities where supported.
9. Use `.env` substitution for shared secrets. Do not bake them into images.

Commands

```
docker build -t quorumlimiter:dev .
docker compose -f deploy/compose.dev.yml config
docker compose -f deploy/compose.dev.yml up -d --build
docker compose -f deploy/compose.dev.yml ps
docker compose -f deploy/compose.dev.yml logs --tail=100
curl -fsS http://localhost:8080/health/live
curl -fsS http://localhost:9090/-/ready
```

To stop without deleting data:

```
docker compose -f deploy/compose.dev.yml down
```

Never add `--volumes` unless you intentionally want to erase the local cluster data.

Files to create or modify

Create `Dockerfile`, `.dockerignore`, `deploy/compose.dev.yml`, `deploy/Caddyfile`, and `deploy/prometheus.yml`. Update `README` quick start and `Makefile` targets.

What the AI coding agent should produce

The agent should produce a deterministic image, valid Compose config, clear health dependencies, no embedded secrets, and comments for exposed ports and volumes. It should inspect the image user and report the final image size.

How to test

Start the stack, locate the leader, create a policy and client, and send decisions. Restart one follower and confirm data survives. Rebuild the image and recreate containers without deleting volumes. Confirm the dashboard and Prometheus still work.

Expected output

`docker compose ps` shows three healthy nodes, healthy Caddy, and healthy Prometheus. Exactly one node reports leader. Caddy can serve public and admin routes, while `http://localhost:8080/internal/raft/status` returns 404 or 403.

Common errors and corrections

- A node that cannot resolve peers usually has URLs using `localhost`. Inside Compose, use service names such as `http://node2:8080`.
- Permission denied on `/data` means the runtime user does not own the mounted directory. Set image directory ownership and volume mount path correctly.
- All nodes sharing one volume corrupts the design. Give each node a distinct named volume.
- Caddy returning 502 during startup may be temporary. Use health checks and retry, but inspect node logs if it persists.

6.12 Phase 12: Integration tests, partitions, restart, and failover scripts

Objective

Prove cluster behavior under concurrency and realistic faults, then package the same demonstrations into reviewer friendly scripts.

Why this phase is necessary

Happy path tests do not establish distributed correctness. This phase targets stale leaders, network partitions, conflicting logs, delayed messages, restarts, and duplicate clients.

Exact actions

1. Build a reusable in process cluster with temporary database files and controllable transport faults.
2. Add tests for initial election, leader stop, follower stop, no quorum, asymmetric partition, delayed messages, duplicate RPCs, stale responses, conflicting follower log, and full restart.
3. Add a concurrency test that sends many decisions for one bucket and verifies the accepted total.
4. Add a model comparison that replays the committed decision log and compares final state.
5. Write `scripts/smoke.sh` to sign in through documented APIs, create a unique policy and client, and send allowed plus denied decisions.
6. Write `scripts/failover.sh` to discover the leader, stop it, wait for a new leader, send a decision, restart the old node, and verify catch up.
7. Make scripts refuse to run if required placeholder secrets or tools are missing.

Commands

```
go test ./test/integration -count=1 -timeout=90s
go test ./test/integration -count=10 -timeout=10m
go test -race ./... -count=1 -timeout=10m
bash scripts/smoke.sh
bash scripts/failover.sh
```

Files to create or modify

Create all `internal/testcluster` and `test/integration` files shown in Section 5. Create `scripts/smoke.sh` and `scripts/failover.sh`. Update Makefile with `integration`, `smoke`, and `failover` targets.

What the AI coding agent should produce

The agent should produce tests with bounded polling helpers rather than fixed long sleeps. On failure, tests should print node roles, terms, log bounds, commit indexes, and applied indexes. Scripts should be strict Bash with `set -euo pipefail`.

How to test

Run the suite repeatedly. Intentionally break a commit rule in a temporary branch and confirm a safety test catches it. Restore the correct code. Run failover while watching logs and Prometheus.

Expected output

The test suite passes repeatedly without flakes. The failover script reports old leader, new leader, election duration, successful post failure decision, and caught up indexes.

Common errors and corrections

- Flaky tests often depend on fixed sleeps. Poll a condition until a clear deadline and print diagnostics on timeout.
- A symmetric test network may miss asymmetric partitions. Control each directed connection separately.
- Docker pause simulates frozen processes, while disconnecting a network simulates communication loss. Use both and name the scenario accurately.
- A test that only checks response counts may miss state divergence. Compare committed logs and exported state.

6.13 Phase 13: Load, performance, security, and quality gates

Objective

Measure the working cluster, establish modest repeatable thresholds, inspect race and security behavior, and create one final local release gate.

Why this phase is necessary

Performance without correctness is not useful, and correctness without measurement leaves operational questions unanswered. The goal is transparent evidence rather than exaggerated throughput claims.

Exact actions

1. Create a k6 scenario with separate subjects and one hot subject.
2. Require explicit API key and policy environment variables.
3. Track allowed, denied, 503, and unexpected responses separately.
4. Start with 25 virtual users for 60 seconds, then raise load carefully.
5. Set initial thresholds of less than 1 percent unexpected responses and p95 below 250 milliseconds on the documented development machine.
6. Add a leader failure scenario at the midpoint and measure recovery separately.
7. Run format, vet, unit, integration, race, lint, vulnerability, secret, image, and header checks.

- Record test hardware, command, commit, request mix, and results in `docs/demo.md`.

Commands

```
go fmt ./...
git diff --exit-code -- '*.go'
go vet ./...
go test ./... -count=1
go test -race ./... -count=1 -timeout=10m
golangci-lint run ./...
govulncheck ./...

docker run --rm --network host \
  -e BASE_URL=http://127.0.0.1:8080 \
  -e API_KEY='qlk_REPLACE_WITH_LOCAL_TEST_KEY' \
  -e POLICY_ID='load_test' \
  -v "$PWD/scripts:/scripts:ro" \
  grafana/k6:2.1.0 run /scripts/load.js
```

Do not place a real production key in shell history. The shown key is a placeholder. For repeated local testing, load a development key from a protected file or temporary environment.

Files to create or modify

Create `scripts/load.js`, `.golangci.yml`, CI compatible quality targets, and performance notes in `docs/demo.md` and `README.md`.

What the AI coding agent should produce

The agent should produce a readable k6 script, conservative thresholds, a failure summary, and commands that exit nonzero on a failed gate. It should not weaken tests to obtain a passing result.

How to test

Run the complete gate twice on a clean stack. Inspect p50, p95, p99, error counts, election recovery time, CPU, memory, database growth, and apply lag. Search the repository and container logs for secret patterns.

Expected output

All correctness and security commands pass. Load results include the exact context and do not claim universal production capacity. During leader failure, a short increase in 503 or latency is documented, followed by recovery.

Common errors and corrections

- k6 cannot reach `localhost` on some Docker setups. On Linux use `--network host`; elsewhere use `host.docker.internal`.
- A high denied count is not an unexpected error if the hot subject deliberately exhausts a bucket. Track it separately.
- Race tests use more memory and time. Reduce integration repetition but never remove the race gate.
- A vulnerability finding requires investigation. Upgrade the affected dependency, confirm tests, and document any unreachable finding rather than suppressing it casually.

6.14 Phase 14: Production style deployment

Objective

Deploy the tested image on an Ubuntu server with a real domain, HTTPS, persistent volumes, firewall rules, backups, monitoring, and a verification procedure.

Why this phase is necessary

A portfolio project is stronger when another person can reach a secure live demonstration. Deployment also reveals assumptions about storage, secrets, networking, process restart, and upgrades.

Exact actions

1. Provision an Ubuntu 24.04 or 26.04 LTS server with at least two virtual CPUs, 4 GB memory, and 40 GB disk for a small demo.
2. Point an A or AAAA DNS record such as `limiter.example.com` to the server.
3. Install Docker from the official repository.
4. Create `/opt/quorumlimiter` and copy only deployment files plus the built image reference.
5. Generate new production secrets on the server. Never reuse development secrets.
6. Set `QL_PRODUCTION=true`, the HTTPS public URL, and persistent named volumes.
7. Change the Caddy site address from local HTTP to the real domain.
8. Allow inbound TCP 22, 80, and 443 only. Bind Prometheus and node debug ports to loopback or do not publish them.
9. Start the stack, inspect certificate issuance, and run the production verification checklist.
10. Schedule encrypted backups of all three node volumes or database exports. Test restore on a separate machine.
11. Document the single host limitation. If three virtual machines are used, allow Raft traffic only over their private network.

Commands

On the server:

```
sudo install -d -m 0750 -o "$USER" -g "$USER" /opt/quorumlimiter
cd /opt/quorumlimiter
umask 077
openssl rand -base64 36
openssl rand -base64 48
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
docker compose -f deploy/compose.prod.yml config
docker compose -f deploy/compose.prod.yml pull
docker compose -f deploy/compose.prod.yml up -d
docker compose -f deploy/compose.prod.yml ps
docker compose -f deploy/compose.prod.yml logs caddy --tail=100
```

From a different computer:

```
curl -i https://limiter.example.com/health/live
curl -i https://limiter.example.com/health/ready
curl -i https://limiter.example.com/internal/raft/status
curl -I https://limiter.example.com/admin/login
```

The internal route must not be publicly usable.

Files to create or modify

Create `deploy/compose.prod.yml`, production Caddy settings, backup instructions, and release notes. Do not commit the production `.env`, certificates, database files, or backup keys.

What the AI coding agent should produce

The agent should produce deployment files with explicit image tags, health checks, restart policies, volumes, safe published ports, production cookies, and comments. It should provide a rollback procedure that restores the previous image without deleting volumes.

How to test

Verify HTTPS, certificate chain, HTTP redirect, security headers, admin login, client decision, denial behavior, Prometheus scrape, leader failover, node restart, data persistence, firewall, backup creation, and a restore rehearsal.

Expected output

The public domain serves HTTPS, admin cookies are Secure, internal and metrics routes are blocked publicly, and the service survives a container restart with data intact.

Common errors and corrections

- Caddy certificate failure commonly means DNS has not propagated or ports 80 and 443 are blocked.
- A redirect loop can occur when public base URL, proxy headers, and Caddy scheme disagree. Set the public URL to HTTPS and trust only Caddy.
- Losing data after recreation means volumes were anonymous or omitted. Use named volumes and inspect them before deployment.
- Three containers on one server do not survive server failure. State this clearly and use the three machine variant if infrastructure resilience is required.

6.15 Phase 15: GitHub packaging and reviewer handoff

Objective

Turn the finished local repository into a clear, reproducible public portfolio artifact without exposing credentials.

Why this phase is necessary

Reviewers often spend only a few minutes on a repository. The README, diagrams, tests, demonstration, and honest limitations must make the technical value visible quickly.

Exact actions

1. Complete the README structure from Section 11.
2. Add architecture and request flow diagrams generated from repository source or exported as clear images.
3. Capture dashboard, policy, client redaction, decision audit, cluster failover, and Prometheus screenshots with no secrets.
4. Add a two minute demonstration script to `docs/demo.md`.
5. Add CI with pinned actions and least privilege permissions.
6. Run a local secret scan and inspect Git history before creating a remote.
7. Make focused local commits with clear messages.
8. Create the remote using GitHub's website or GitHub CLI only when ready. Never give an AI agent a password or personal access token.

9. Push through your configured secure Git credential method.
10. Add repository topics, description, live demo link if safe, and release tag `v1.0.0`.

Commands

Run locally:

```
git status --short
git diff --check
go test ./...
go test -race ./... -count=1 -timeout=10m
golangci-lint run ./...
govulncheck ./...
git grep -nE 'QL_(ADMIN|CLUSTER|SESSION|API_KEY).*=' -- '!*.example'
git log --oneline --decorate -n 15
```

After you create an empty GitHub repository yourself:

```
git remote add origin git@github.com:yourname/quorumlimiter.git
git branch -M main
git push -u origin main
```

Files to create or modify

Finalize `README.md`, `LICENSE`, all docs files, screenshot assets, `.github/workflows/ci.yml`, issue templates if desired, and release notes.

What the AI coding agent should produce

The agent should improve documentation and CI only from verified project behavior. It must not invent benchmark numbers, supported failure modes, or production claims. It should stop before any push and report the exact commits and commands for the user to review.

How to test

Clone the repository into a fresh directory without your `.env`. Follow only the README. Generate new development secrets, start the stack, run smoke and failover scripts, and confirm screenshots match the current interface.

Expected output

A new reviewer can understand the project in two minutes and run it in under ten minutes after prerequisites are installed. CI is green, repository history has no secrets, and limitations are easy to find.

Common errors and corrections

- README commands that work only in the original directory usually depend on uncommitted files. Test a fresh clone.
- Screenshots can leak raw client keys, admin tokens, terminal history, domains, or user identifiers. Inspect every pixel before committing.
- A red CI badge may reflect a different Go version than local. Pin Go 1.26.5 in the workflow.
- If a secret was committed, removing the current file is insufficient. Rotate it immediately and rewrite history with an appropriate tool before publishing.

7. AI Coding Agent Prompts

Use these prompts one at a time from the repository root. Before each prompt, make sure the prior phase is committed or otherwise easy to review. Replace `github.com/yourname/quorumlimiter` only if your module path differs.

Every prompt asks the agent to inspect existing code first, preserve working behavior, and stop after a verified phase. If an agent suggests importing a Raft implementation, using Redis for limiting, skipping durability, or weakening tests, reject that suggestion because it changes the project goal.

7.1 Prompt for Phase 1

You are working in a new repository for QuorumLimiter, a distributed token bucket rate limiter. The module is `github.com/yourname/quorumlimiter`. Use Go 1.26.5. The only planned application dependencies are `bbolt v1.5.0` and `Prometheus client_golang v1.24.1`. Do not add a web framework, Redis, or a Raft library.

Complete only Phase 1: repository scaffold, validated environment configuration, structured logging, process lifecycle, and `GET /health/live`. Read the current `go.mod` and existing files before editing.

You may create or modify:

```
cmd/quorumlimiter/main.go
internal/config/*
internal/server/*
docs/adr/0001-fixed-three-node-raft.md
docs/adr/0002-deterministic-token-bucket.md
docs/adr/0003-bbolt-storage.md
.gitignore, .dockerignore, .env.example, Makefile, README.md
```

Config must include node ID, bind address, advertised URL, exactly three peers, data path, election min and max, heartbeat interval, proposal timeout, public base URL, production flag, log level, cluster token, admin token, session key, and API key pepper. Validate peer uniqueness and timeout relationships. Never log secret values. Use `log/slog` JSON output. Handle `SIGINT` and `SIGTERM` with a five second graceful shutdown context.

Add table driven config tests. Do not create consensus behavior yet. Run:

```
gofmt on changed Go files
go test ./...
go vet ./...
```

Return a completion report with files changed, key decisions, commands and their results, manual health check command, and any remaining Phase 1 risks. Stop after Phase 1.

7.2 Prompt for Phase 2

Continue QuorumLimiter using Go 1.26.5 and bbolt v1.5.0. Complete only Phase 2: durable local storage and schema migration. Inspect existing configuration, shutdown, and tests first.

You may create or modify:

- internal/storage/*
- cmd/quorumlimiter/main.go
- internal/server/*
- docs/architecture.md
- Makefile

Open one bbolt file with mode 0600, create its parent directory with mode 0700, and use a one second open timeout. Implement an idempotent schema version 1 migration. Define private buckets for meta, raft_log, policies, clients, client_prefix, token_buckets, idempotency, and audit. Store uint64 keys and values in fixed eight byte big endian format.

Implement typed methods for current term, voted for, log lookup and ranges, append, suffix truncation, first and last index, commit checkpoint, and applied checkpoint. Add safe read methods for application state. Keep application mutation ownership restricted to a state machine transaction callback. Copy bbolt byte slices before returning from a transaction. Wrap errors with operation context and close the store on process shutdown.

Add tests for migration, reopen recovery, log ordering, truncation, rollback, missing values, concurrent reads, and file permissions. Use t.TempDir. Do not add Raft elections or HTTP mutation endpoints.

Run gofmt, go test ./internal/storage -count=1, go test -race ./internal/storage -count=1, and go test ./....

Report files changed, schema created, durability guarantees, test results, and limitations. Stop after Phase 2.

7.3 Prompt for Phase 3

Continue QuorumLimiter. Complete only Phase 3: the deterministic token bucket state machine. Use Go 1.26.5, bbolt v1.5.0, and the existing storage layer.

You may create or modify:

```
internal/limiter/*
internal/storage/state_store.go
internal/storage/*_test.go
docs/adr/0002-deterministic-token-bucket.md
docs/architecture.md
```

Create versioned command envelopes for policy creation and update, policy activation, API client creation and revocation, decision, and deterministic pruning. Define models from the architecture document. Store balances in milli tokens. Preserve integer division remainder. Use the command timestamp, never time.Now during apply. Use effective time equal to the maximum of command time and prior refill time. Check arithmetic overflow and clamp to capacity. Perform idempotency lookup before refill or deduction. Store and return the original result for duplicates. When a policy changes, deterministically visit its existing buckets, refill them under the old policy through the update timestamp, clamp to new capacity, reset an incompatible remainder, and record the new policy version before storing the new policy in the same transaction.

The apply path must not use floating point, random numbers, goroutines, network calls, environment variables, or local clocks. Add exhaustive table tests plus a fixed seed randomized reference model test. Add a test that applies the same serialized commands to two fresh stores and compares normalized state.

Do not add elections, replication, public handlers, or dashboard code.

```
Run gofmt, go test ./internal/limiter -count=1, go test
./internal/limiter -run TestTokenBucket -count=100, go test -race
./internal/limiter, and go test ./...
```

Report files, command formats, arithmetic choices, test results, and any documented bounds. Stop after Phase 3.

7.4 Prompt for Phase 4

Continue QuorumLimiter. Complete only Phase 4: Raft types, persistent transitions, recovery, a serialized event loop, and immutable status. Use Go 1.26.5. Implement consensus code yourself. Do not import hashicorp/raft, etcd/raft, Dragonboat, Redis, or another consensus implementation.

You may create or modify:

```
internal/raft/types.go
internal/raft/node.go
internal/raft/rpc.go
internal/raft/*_test.go
internal/storage/raft_store.go
docs/architecture.md
```

Define follower, candidate, and leader roles. Define versioned RequestVote and AppendEntries messages with cluster ID, source node, term, and log metadata. Create narrow interfaces for persistence, transport, deterministic apply, and timers. Recover persistent state on construction. One event loop must own all mutable Raft state. Implement legal state transition helpers and higher term step down that persists the new term and cleared vote before responding. Provide status queries through the event loop or immutable snapshots.

Add deterministic fake clock tests for recovery, term changes, vote clearing, legal transitions, concurrent status reads, and clean shutdown. Do not implement a complete election or replication yet.

```
Run gofmt, go test ./internal/raft -count=1, go test -race ./internal/raft,
and go test ./...
```

Report the concurrency ownership model, persistent versus volatile state, files, tests, and unfinished Raft behavior. Stop after Phase 4.

7.5 Prompt for Phase 5

Continue QuorumLimiter. Complete only Phase 5: leader election and RequestVote. Use Go 1.26.5 and the existing Raft event loop. Consensus must remain an original implementation with fixed three node membership.

You may create or modify:

```
internal/raft/election.go
internal/raft/node.go
internal/raft/rpc.go
internal/raft/*_test.go
internal/api/internal_raft.go
internal/api/routes.go
docs/architecture.md
```

Implement randomized election timeouts from 800 to 1400 ms through the injectable timer source. On timeout persist incremented term and self vote, then send RequestVote to peers. Grant one vote per term only when the candidate's log is at least as up to date, comparing last term before last index. Count only current term votes. A majority is two nodes. On election, initialize nextIndex and matchIndex and append a current term no operation entry. Step down on every higher term request or response.

Protect internal HTTP RPC with the cluster bearer token, a source node allowlist, content type validation, and a 64 KiB body limit. Do not expose the route through a public gateway config yet. Avoid arbitrary sleeps in unit tests.

Test stale candidates, split vote, higher term, stale response, restart after voting, one leader per term, leader loss, and old leader restart.

Run gofmt, focused election tests with -count=25, race tests with -count=10, and go test ./....

Report files, election rules, test evidence, and known missing replication work. Stop after Phase 5.

7.6 Prompt for Phase 6

Continue QuorumLimiter. Complete only Phase 6: AppendEntries replication, conflict repair, majority commit, ordered state machine apply, proposal waiters, and restart replay. Use Go 1.26.5 and fixed three node membership.

You may create or modify:

```
internal/raft/replication.go
internal/raft/apply.go
internal/raft/waiters.go
internal/raft/node.go
internal/raft/rpc.go
internal/raft/*_test.go
internal/storage/raft_store.go
test/integration/cluster_test.go
test/integration/recovery_test.go
internal/testcluster/*
docs/architecture.md
```

Append a leader proposal locally before transmission. Implement previous index and term matching, transactional follower suffix truncation plus append, conflict term hints, per follower nextIndex and matchIndex, and retries. Advance commit only for a majority replicated entry from the current term. Apply every index in order. Commit one entry's application writes and its new last_applied value in the same bbolt transaction. Return a proposal result only after its entry is committed and applied. Bound and clean the waiter registry. A caller timeout must not undo an entry that may commit. Replay committed but unapplied entries before readiness on restart.

Add tests for normal replication, one follower unavailable, isolated leader, conflicting suffix, stale responses, old term commit restriction, ordered apply, timeout cleanup, restart, and convergence. Use bounded condition waits.

Run focused tests repeatedly, all integration tests, race tests, and go test ./.... Report changed files, safety rules, test results, and remaining API work. Stop after Phase 6.

7.7 Prompt for Phase 7

Continue QuorumLimiter. Complete only Phase 7: the public decision API, API key authentication, follower to leader forwarding, stable errors, and a small Go client. Use Go 1.26.5, bbolt v1.5.0, and existing consensus interfaces.

You may create or modify:

```
internal/api/decision.go
internal/api/types.go
internal/api/errors.go
internal/api/middleware.go
internal/api/routes.go
internal/auth/apikey.go
internal/auth/apikey_test.go
pkg/client/*
test/integration/idempotency_test.go
docs/api.md
```

Implement POST /v1/decisions. Require bearer API key, Idempotency-Key, and JSON content type. Limit body to 64 KiB, reject unknown fields, and validate policy, subject, cost, and request ID. Authenticate using key prefix lookup and HMAC SHA 256 with QL_API_KEY_PEPPER, then constant time compare.

If the node is a follower, forward to its known leader through the private transport. Preserve authorization and idempotency, strip hop by hop headers, add a one hop marker, and prevent loops. Return 503 with Retry-After when no leader is known. The leader proposes and waits for committed apply. Define stable JSON for 200, 400, 401, 404, 409, 415, 422, 429, 503, and 504.

Add handler, forwarding, integration, and client retry tests. The client may retry only when an idempotency key exists. Never log keys or full subjects.

Run gofmt, package tests, integration tests with -count=5, race tests, and all tests. Report endpoint behavior, files, tests, and retry semantics. Stop after Phase 7.

7.8 Prompt for Phase 8

Continue QuorumLimiter. Complete only Phase 8: admin authentication, signed sessions, CSRF defense, login throttling, secure headers, and Raft backed admin JSON mutations. Use Go 1.26.5 and standard library cryptography.

You may create or modify:

```
internal/auth/session.go
internal/auth/csrf.go
internal/auth/*_test.go
internal/api/admin.go
internal/api/middleware.go
internal/api/routes.go
internal/api/*_test.go
internal/limiter/command.go
internal/limiter/state_machine.go
internal/storage/state_store.go
docs/security.md
```

Implement admin login and logout. Compare QL_ADMIN_TOKEN in constant time. Create an eight hour signed HttpOnly SameSite Strict session with issue time, expiry, and random CSRF value. Set Secure when QL_PRODUCTION is true. Require same origin and CSRF token for mutations. Limit failed sign ins to five per IP per minute in a bounded in memory structure.

Implement policy create, update, activation, client create, and revoke as Raft commands. Generate raw qk keys on the leader with crypto/rand. Replicate only prefix and HMAC digest and return the raw key once after commit. Never persist or log the raw key. Add redacted admin audit events.

Test modified and expired cookies, CSRF failures, origin failures, login throttle, validation, duplicate IDs, key creation, authentication, revocation, and secret absence from logs.

Run gofmt, focused tests repeatedly, race tests, integration lifecycle tests, and all tests. Report files, security behavior, tests, and residual risks. Stop after Phase 8.

7.9 Prompt for Phase 9

Continue QuorumLimiter. Complete only Phase 9: the full server rendered admin dashboard. Use Go 1.26.5 html/template, HTML5, CSS, and minimal vanilla JavaScript. Do not add React, Vue, Tailwind, Bootstrap, npm, or CDN assets.

You may create or modify:

```
internal/dashboard/*
internal/api/routes.go
internal/api/admin.go
internal/api/*_test.go
internal/server/*
```

Build login, overview, policies, API clients, decisions, cluster, and API docs pages exactly as described in docs. Embed templates and assets into the binary. Use semantic HTML, explicit labels, validation summaries, keyboard focus, responsive Grid and Flexbox, and accessible status text. JavaScript may only enhance copy, confirm, disclosure, and polling. All critical forms must work without JavaScript.

Never place a real key in source or docs. Show a newly created client key only in the immediate no-store response. Escape all user data. Avoid template.HTML. Include empty, success, error, and unavailable states. Do not add destructive cluster controls.

Add handler and rendering tests for auth redirects, CSRF fields, controls, escaping, cache headers, and embedded assets. Run gofmt, dashboard and API tests, race tests, and all tests.

Report files, routes and screens completed, accessibility checks, test results, and manual viewport checks still required. Stop after Phase 9.

7.10 Prompt for Phase 10

Continue QuorumLimiter. Complete only Phase 10: health, readiness, structured observability, and Prometheus metrics. Use `client_golang` v1.24.1 and Prometheus 3.13.1 configuration.

You may create or modify:

```
internal/metrics/*
internal/api/middleware.go
internal/api/routes.go
internal/raft/*
internal/server/*
deploy/prometheus.yml
docs/architecture.md
docs/security.md
```

Keep liveness simple. Readiness must distinguish leader quorum contact, follower leader knowledge, catchup lag, startup replay, and shutdown. Add bounded label metrics for request count and latency, decisions, role, term, elections, RPCs, log and apply indexes, proposal latency, database size, and last quorum contact. Never label by API key, request ID, subject, client ID, or raw path.

Add structured logs for role and term changes, vote grants, leader changes, commit advancement, recovery, and shutdown. Redact authorization, cookies, subjects, and one time key responses. Avoid logging every heartbeat.

Use injectable Prometheus registries in tests. Add tests for readiness changes, metric values, bounded cardinality, duplicate registration, and secret absence. Run `gofmt`, focused tests, race tests, and all tests.

Report metrics and labels, readiness rules, log events, files, tests, and recommended initial alerts. Stop after Phase 10.

7.11 Prompt for Phase 11

Continue QuorumLimiter. Complete only Phase 11: reproducible Docker and local three node Compose deployment. Use Go 1.26.5, Docker Engine 29.6.1 compatible syntax, Docker Compose 5.3.0, Caddy 2.11.4, and Prometheus 3.13.1.

You may create or modify:

```
Dockerfile
.dockerignore
deploy/compose.dev.yml
deploy/Caddyfile
deploy/prometheus.yml
Makefile
README.md
```

Use a multi stage `golang:1.26.5-alpine3.23` build and `alpine:3.23` runtime. Produce a static nonroot binary with `trimp` path. Configure three services `node1`, `node2`, `node3` with service name peer URLs, separate named volumes, health checks, restart policies, resource limits, stop grace periods, dropped capabilities, and read only roots where practical. Shared secrets must come from an ignored `.env`.

Expose Caddy on 8080. Bind node debug and Prometheus ports to 127.0.0.1 only. Caddy may route public and admin endpoints but must block `/internal` and `/metrics`. Pin `caddy:2.11.4-alpine` and `prom/prometheus:v3.13.1`.

Run `docker build`, `docker compose config`, `docker compose up`, health checks, and basic `curl` checks. Do not delete volumes. Report files, image size and user, service health, ports, volumes, commands, and unresolved issues. Stop after Phase 11.

7.12 Prompt for Phase 12

Continue QuorumLimiter. Complete only Phase 12: multi node integration tests, fault controls, smoke script, and failover demonstration.

You may create or modify:

```
internal/testcluster/*
test/integration/*
scripts/smoke.sh
scripts/failover.sh
Makefile
docs/demo.md
```

Build an in process three node cluster with real loopback HTTP listeners, temporary bbolt files, injectable clocks where appropriate, and a directed fault transport that can drop, delay, duplicate, and heal messages. Use bounded polling, not fixed long sleeps. On failure print roles, terms, log bounds, commit indexes, applied indexes, and peer progress.

Test initial election, leader stop, follower stop, no quorum, asymmetric partition, delayed and duplicate RPCs, stale messages, conflict repair, full restart, catchup, duplicate decision, and concurrent hot bucket acceptance. Compare replayed committed state with actual state.

Write strict Bash smoke and failover scripts. They must validate prerequisites, avoid printing secrets, discover the leader, measure election recovery, verify a post failure decision, restart the old node, and confirm catchup.

Run integration tests repeatedly, full race tests, smoke, and failover. Report files, scenarios, commands and results, timing, and any flaky behavior. Stop after Phase 12.

7.13 Prompt for Phase 13

Continue QuorumLimiter. Complete only Phase 13: load testing, security checks, lint configuration, and a reproducible local release gate. Use k6 2.1.0, golangci-lint 2.12.2, and govulncheck 1.1.4.

You may create or modify:

```
scripts/load.js
.golangci.yml
Makefile
docs/demo.md
docs/security.md
README.md
.github/workflows/ci.yml only if needed for local command parity
```

Create k6 scenarios for many independent subjects and one hot subject. Separate allowed, denied, 503, and unexpected outcomes. Require BASE_URL, API_KEY, and POLICY_ID from environment. Set initial thresholds below 1 percent unexpected responses and p95 below 250 ms, with a note that results depend on hardware. Add a documented leader failure measurement.

Create one release-check target that runs formatting check, go vet, tests, race tests, golangci-lint, govulncheck, and a local secret pattern scan. Do not suppress findings merely to pass. Never embed test keys.

Run all quality commands and one baseline k6 run. Record exact hardware, commit, duration, virtual users, request mix, latency, errors, and failover effect. Report files, commands, results, threshold rationale, findings, and remaining work. Stop after Phase 13.

7.14 Prompt for Phase 14

Continue QuorumLimiter. Complete only Phase 14: production style Compose deployment documentation and files. Use explicit image tags, Caddy 2.11.4, and Prometheus 3.13.1. Do not perform an external deployment or request credentials.

You may create or modify:

- deploy/compose.prod.yml
- deploy/Caddyfile
- docs/deployment.md
- docs/security.md
- README.md
- Makefile

Create a production Compose file for Ubuntu with three distinct persistent node volumes, Caddy as the only public service, loopback or private Prometheus, health checks, restart policies, nonroot execution, read only roots where possible, dropped capabilities, resource limits, graceful stop, and QL_PRODUCTION=true. All secrets must be environment references, never values. Use a DOMAIN placeholder and explain DNS plus ports 80 and 443.

Document first deployment, verification, firewall, logs, backup, restore rehearsal, image upgrade, rollback without volume deletion, and the limitation of three containers on one host. Add a separate three virtual machine variant using private peer addresses and firewall rules. Do not claim dynamic membership.

Validate Compose config with placeholder environment values and run existing tests. Report files, config validation, exposed ports, volumes, backup and rollback steps, and manual actions the user must perform. Stop after Phase 14.

7.15 Prompt for Phase 15

Continue the completed QuorumLimiter project. Complete only Phase 15: repository documentation, screenshots checklist, CI, and reviewer handoff. Do not push, publish a release, create a remote, or request GitHub credentials.

Use Go 1.26.5, golangci-lint 2.12.2, govulncheck 1.1.4, Docker compatible with 29.6.1, Compose 5.3.0, k6 2.1.0, Caddy 2.11.4, Prometheus 3.13.1, bbolt 1.5.0, and client_golang 1.24.1.

You may create or modify:

- README.md
- LICENSE
- docs/*
- .github/workflows/ci.yml
- .github/ISSUE_TEMPLATE/* if useful
- repository diagram and screenshot assets

Write the README structure from the guide. Include quick start, architecture, request flow, features, API example with placeholders, tests, failover demo, metrics, deployment link placeholder, design decisions, explicit limitations, roadmap, and license. Do not invent benchmark results. Add a least privilege CI workflow pinned to trusted action versions and Go 1.26.5.

Create a screenshot checklist that explicitly forbids raw keys, tokens, cookies, user data, and terminal history. Run formatting check, tests, race tests, linting, vulnerability scan, diff check, and secret pattern scans. Inspect Git status and recent history.

Return files changed, documentation sections, CI jobs, every command result, remaining manual screenshots, and exact safe commands the user may run later to add a remote and push. Stop before all external writes.

8. Database and API Details

8.1 Database overview

There is no shared database server. Every node owns one bbolt file, normally `/data/quorumlimiter.db` in its container. Raft metadata and log entries can differ temporarily while a follower catches up. Applied application state must converge once nodes share the same commit index.

bbolt calls its top level key spaces buckets. All writes are transactional. A failed transaction makes none of its changes visible. Keys and values are byte arrays, so `internal/storage/encoding.go` is the single place that defines numeric and composite encodings.

8.2 Bucket schema

Bucket	Key	Value	Written by
meta	Fixed names such as <code>schema_version</code> , <code>current_term</code> , <code>voted_for</code> , <code>commit_index</code> , <code>last_applied</code> , <code>cluster_id</code>	Big endian integers or UTF 8 bytes	Migration and Raft
raft_log	Eight byte big endian log index	Encoded <code>LogEntry</code>	Raft
policies	Policy ID bytes	Encoded <code>Policy</code>	Applied state machine
clients	Client ID bytes	Encoded <code>Client</code>	Applied state machine
client_prefix	Public key prefix bytes	Client ID bytes	Applied state machine
token_buckets	Length prefixed policy ID plus subject	Encoded <code>TokenBucketState</code>	Applied state machine
idempotency	Length prefixed client ID plus request ID	Encoded <code>DecisionRecord</code>	Applied state machine
audit	Eight byte big endian log index	Encoded <code>AuditEvent</code>	Applied state machine

Values may use JSON in version 1 because it is inspectable and adequate for the portfolio scale. Each value includes a `schema_version` field. Do not encode Go interface type names. If performance later matters, migrate to a stable binary format through an explicit schema version.

Composite keys must be unambiguous. Do not concatenate strings with a character that may appear in a subject. Encode each part as a two byte big endian length followed by bytes:

```
[policy length][policy bytes][subject length][subject bytes]
```

Reject parts longer than 65,535 bytes before encoding. Current validation is much smaller.

8.3 Raft metadata and log entry

Field	Type	Validation and meaning
schema_version	uint64	Must equal a supported local version
cluster_id	string	Required fixed identifier shared by three nodes
current_term	uint64	Monotonic, persisted before term dependent response
voted_for	string	Empty or one configured node ID for current term
commit_index	uint64	Never decreases and never exceeds last log index
last_applied	uint64	Never decreases and never exceeds commit index

LogEntry fields:

Field	Type	Validation and meaning
index	uint64	Positive and contiguous after the sentinel entry
term	uint64	Term in which the leader created the entry
kind	string	noop or command
command	byte array	Empty for no operation, otherwise a versioned command envelope

An optional sentinel at index zero and term zero simplifies previous entry checks. It is not an application command and is never shown in audits.

8.4 Policy model

Field	Type	Validation
schema_version	uint32	1
id	string	Regex <code>^[a-z][a-z0-9_]{2,47}\$</code>
name	string	Trimmed length 3 through 64
capacity_tokens	int64	1 through 1,000,000
refill_tokens	int64	1 through 1,000,000
refill_interval_ms	int64	1,000 through 86,400,000
max_cost_tokens	int64	1 through capacity
active	bool	New policies normally true
version	uint64	Starts at 1 and increments on update
created_at_ms	int64	Positive leader supplied Unix milliseconds
updated_at_ms	int64	At least created time and monotonic

Example stored value:

```
{
  "schema_version": 1,
  "id": "password_reset",
  "name": "Password reset emails",
  "capacity_tokens": 5,
  "refill_tokens": 1,
  "refill_interval_ms": 12000,
  "max_cost_tokens": 1,
  "active": true,
  "version": 1,
  "created_at_ms": 1785312000000,
  "updated_at_ms": 1785312000000
}
```

8.5 API client model

Field	Type	Validation
schema_version	uint32	1
id	string	Server generated cli_ plus base32 random ID
name	string	Trimmed length 3 through 80
key_prefix	string	Eight base32 characters, unique
key_digest	32 bytes	HMAC SHA 256 of the complete raw key
allowed_policy_ids	string array	One through 100 unique policy IDs, or exactly *
active	bool	True on creation, false after revocation
created_at_ms	int64	Positive leader timestamp
revoked_at_ms	int64 or null	Null while active

Example display representation, which deliberately omits the digest:

```
{
  "id": "cli_3M7WX6C2S9",
  "name": "Account service",
  "key_prefix": "K4N9Q2PT",
  "allowed_policy_ids": ["password_reset"],
  "active": true,
  "created_at": "2026-07-29T08:00:00Z",
  "revoked_at": null
}
```

8.6 Token bucket state

Field	Type	Validation and meaning
schema_version	uint32	1
policy_id	string	Existing policy at apply time
subject	string	1 through 128 characters matching the subject rule
tokens_milli	int64	Zero through policy capacity times 1000
last_refill_ms	int64	Monotonic effective command time
refill_remainder	int64	Zero through refill interval minus one
policy_version	uint64	Policy version used at last calculation
updated_log_index	uint64	Log index of last nonduplicate decision

When a policy is edited, the committed update command iterates existing buckets for that policy. Each bucket is refilled under the old policy only through the update timestamp, clamped to the new capacity, assigned the new policy version, and given a reset remainder if the refill interval changed. The policy is stored after conversion in the same transaction. This deterministic version 1 approach can make updates slow for a policy with many subjects, so the dashboard must warn the administrator and metrics must measure apply duration.

8.7 Decision and idempotency record

Field	Type	Validation and meaning
schema_version	uint32	1
request_id	string	16 through 128 URL safe characters
client_id	string	Authenticated client
policy_id	string	Requested policy
subject	string	Stored for bucket identity; redacted in normal UI
cost_tokens	int64	1 through policy maximum cost
allowed	bool	Deterministic result
remaining_milli	int64	Balance after the decision
retry_after_ms	int64	Zero when allowed, positive estimate when denied
observed_at_ms	int64	Timestamp from command
expires_at_ms	int64	Idempotency retention cutoff
leader_term	uint64	Term of the log entry
log_index	uint64	Committed entry index

Retain idempotency records for 24 hours in the first version. A replicated prune command removes expired records based on its own timestamp. Never let each node prune on its local clock.

8.8 Audit event

Audit records use the committed log index as a naturally ordered key. Fields are schema version, event type, timestamp, actor type, actor ID, request ID, policy ID, redacted or hashed subject display, allowed result, cost, remaining balance, term, log index, and safe metadata. Do not include raw keys, tokens, cookies, complete authorization headers, or full request bodies.

Keep the most recent 100,000 audit records in the first version. A deterministic prune command removes older indexes only after a newer committed command establishes the cutoff.

8.9 Migration instructions

On every startup:

1. Open the database.
2. Begin one writable transaction.
3. Read `schema_version`. Missing means version zero.
4. Apply each migration in ascending order.
5. Write the new version only after that migration succeeds.
6. Commit the transaction.
7. Refuse startup if the stored version is newer than the binary supports.

Schema version 1 creates all buckets and the index zero sentinel. Migration tests must open a version zero database, apply version 1, close, reopen, and apply migration again without changes.

Before a future schema upgrade, back up one caught up follower, test the new binary against a copy, then roll followers before the leader. A schema change that affects replicated command interpretation also needs a protocol compatibility plan. Do not mix binaries that apply the same command differently.

8.10 API conventions

All JSON uses UTF 8. Timestamps in public responses use RFC 3339 UTC. Stored command timestamps use Unix milliseconds. Error responses use:

```
{
  "error": {
    "code": "invalid_request",
    "message": "cost must be between 1 and the policy maximum",
    "request_id": "req_01JEXAMPLE"
  }
}
```

The `message` is safe for a user. Logs may contain a more detailed internal error with secrets redacted. Every response receives `X-Request-ID`. Clients may supply a valid request ID header, or the server generates one. The idempotency key is a separate logical operation identifier.

8.11 Endpoint inventory

Method	Path	Purpose	Authentication
GET	/health/live	Process liveness	None
GET	/health/ready	Consensus readiness	None
GET	/metrics	Prometheus metrics	Private network only
POST	/v1/decisions	Consume or reject tokens	Client bearer key
GET	/admin/login	Render sign in	None
POST	/admin/login	Create admin session	Admin token form
POST	/admin/logout	Clear session	Admin session and CSRF
GET	/admin	Overview page	Admin session
GET	/admin/policies	Policies page	Admin session
POST	/admin/policies	Create policy form	Admin session and CSRF
POST	/admin/policies/{id}/update	Update policy form	Admin session and CSRF
POST	/admin/policies/{id}/activation	Activate or deactivate	Admin session and CSRF
GET	/admin/clients	Clients page	Admin session
POST	/admin/clients	Create client form	Admin session and CSRF
POST	/admin/clients/{id}/revoke	Revoke client	Admin session and CSRF
GET	/admin/decisions	Audit page	Admin session
GET	/admin/cluster	Cluster page	Admin session
GET	/admin/docs	Examples page	Admin session
POST	/admin/test-decisions	Dashboard test decision	Admin session and CSRF
GET	/api/admin/overview	Overview JSON	Admin session
GET	/api/admin/policies	List policy JSON	Admin session
POST	/api/admin/policies	Create policy JSON	Admin session and CSRF
PUT	/api/admin/policies/{id}	Update policy JSON	Admin session and CSRF
POST	/api/admin/policies/{id}/activation	Change active status	Admin session and CSRF
GET	/api/admin/clients	List clients	Admin session
POST	/api/admin/clients	Create client	Admin session and CSRF
POST	/api/admin/clients/{id}/revoke	Revoke client	Admin session and CSRF
GET	/api/admin/decisions	Query audits	Admin session
GET	/api/admin/cluster	Query node state	Admin session
POST	/api/admin/test-decisions	Admin decision test	Admin session and CSRF
POST	/internal/raft/request-vote	Election RPC	Cluster bearer token
POST	/internal/raft/append-entries	Replication RPC	Cluster bearer token
GET	/internal/raft/status	Peer diagnostics	Cluster bearer token

8.12 Health and metrics endpoints

GET /health/live returns 200 whenever the HTTP process is running:

```
{ "status": "alive", "node_id": "node1" }
```

GET /health/ready returns 200 when the node can safely participate and 503 otherwise:

```
{
  "status": "ready",
  "node_id": "node2",
  "role": "follower",
  "term": 18,
  "leader_id": "node1",
  "commit_index": 842,
  "last_applied": 842
}
```

Possible readiness codes are `starting`, `replaying`, `no_leader`, `no_quorum`, `lagging`, `shutting_down`, and `ready`. These endpoints never disclose peer tokens or full addresses.

GET /metrics returns Prometheus text and is not public. It may return 401 or 404 through Caddy even though direct private scrapes succeed.

8.13 Public decision endpoint

Request

POST /v1/decisions

Required headers:

```
Authorization: Bearer qlk_<prefix>_<secret>
Idempotency-Key: 16 to 128 URL safe characters
Content-Type: application/json
```

Body:

```
{
  "policy_id": "password_reset",
  "subject": "customer_4821",
  "cost": 1
}
```

`policy_id` follows the policy ID regex. `subject` is 1 through 128 characters and matches `^[A-Za-z0-9._:@/\-]+$`. `cost` is a positive whole number no greater than policy maximum.

Allowed response

Status 200:

```
{
  "request_id": "01JEXAMPLE000000000000000000",
  "policy_id": "password_reset",
  "allowed": true,
  "cost": 1,
  "remaining_milli": 4000,
  "retry_after_ms": 0,
  "observed_at": "2026-07-29T08:00:00Z",
  "leader_term": 18,
  "log_index": 843,
  "duplicate": false
}
```

Denied response

Status 429 with integer Retry-After seconds:

```
{
  "request_id": "01JEXAMPLE000000000000000001",
  "policy_id": "password_reset",
  "allowed": false,
  "cost": 1,
  "remaining_milli": 220,
  "retry_after_ms": 9360,
  "observed_at": "2026-07-29T08:00:03Z",
  "leader_term": 18,
  "log_index": 849,
  "duplicate": false
}
```

Error responses

Status	Code	Situation
400	invalid_json	Malformed JSON or unknown field
401	invalid_api_key	Missing, malformed, unknown, or revoked key
404	policy_not_found	Policy does not exist
409	idempotency_conflict	Same client and idempotency key used with different request content
415	unsupported_media_type	Content type is not JSON
422	validation_failed	Invalid policy ID, subject, cost, or idempotency key
429	rate_limited	Valid request denied by token bucket
503	leader_unavailable	No leader or no quorum
504	proposal_timeout	Client wait ended before a known result

A 504 caller must retry the same logical request with the same idempotency key. Using a new key could charge twice if the first command later commits.

8.14 Admin sign in and session endpoints

GET /admin/login returns HTML and a login form. An active valid session redirects to /admin.

POST /admin/login accepts application/x-www-form-urlencoded with field admin_token. Success returns 303 See Other to /admin and sets the signed cookie. Failure returns 401 with the same generic page. Excess attempts return 429.

POST /admin/logout requires a valid session, CSRF form field, and same origin. It expires the cookie and returns 303 to the login page.

Admin HTML GET routes return 200, 302 to login if unauthenticated, or 503 when the required cluster data cannot be read. Form mutation routes return 303 after success so refreshing the browser does not repeat a mutation. Validation failure returns 422 and the same form with escaped values and field messages.

8.15 Policy administration API

List policies

GET /api/admin/policies?status=active&q=reset&limit=100

Status 200 returns a JSON array with policy fields. `limit` is 1 through 200. Invalid query values return 422.

Create policy

POST `/api/admin/policies` requires session, same origin, and X-CSRF-Token.

```
{
  "id": "password_reset",
  "name": "Password reset emails",
  "capacity_tokens": 5,
  "refill_tokens": 1,
  "refill_interval_ms": 12000,
  "max_cost_tokens": 1,
  "active": true
}
```

Success returns 201 and the committed policy. Duplicate ID returns 409. Validation returns 422. No leader or quorum returns 503. Proposal wait timeout returns 504.

Update policy

PUT `/api/admin/policies/password_reset`

Body contains editable fields plus required `expected_version`. The ID cannot change.

```
{
  "name": "Password reset emails",
  "capacity_tokens": 8,
  "refill_tokens": 1,
  "refill_interval_ms": 12000,
  "max_cost_tokens": 1,
  "expected_version": 1
}
```

Success returns 200 with version 2. A stale expected version returns 409 `version_conflict`. Missing policy returns 404.

Change activation

POST `/api/admin/policies/password_reset/activation`

```
{"active": false, "expected_version": 2}
```

Success returns 200. It does not delete bucket or audit data.

8.16 Client administration API

List clients

GET `/api/admin/clients?status=active&limit=100` returns safe display objects without digests or raw keys.

Create client

POST `/api/admin/clients`

```
{
  "name": "Account service",
  "allowed_policy_ids": ["password_reset"]
}
```

Success returns 201:

```
{
  "client": {
    "id": "cli_3M7WX6C2S9",
    "name": "Account service",
    "key_prefix": "K4N9Q2PT",
    "allowed_policy_ids": ["password_reset"],
    "active": true
  },
  "api_key": "qlk_K4N9Q2PT_REPLACE_EXAMPLE_ONLY"
}
```

The real key is shown once. The response has `Cache-Control: no-store`. Errors include 404 for an allowed policy that does not exist, 409 for name or idempotency conflict if enforced, 422, 503, and 504.

If client creation returns 504, the command may still commit after the raw key response is lost. The administrator should list clients, revoke the unusable new client if it appears, and create another. Never make the raw key recoverable by storing it in plaintext.

Revoke client

POST /api/admin/clients/cli_3M7WX6C2S9/revoke

```
{"confirm":true}
```

Success returns 200 and safe client state. An already revoked client returns 200 with `already_revoked: true`, which makes retry safe. Unknown client returns 404.

8.17 Audit, overview, cluster, and admin test APIs

GET /api/admin/overview returns safe cluster summary and recent aggregated counters. It does not return raw secrets or complete subjects.

GET /api/admin/decisions accepts `result`, `policy_id`, `client_id`, `request_id`, `subject`, `before_index`, and `limit`. Limit is 1 through 200. Normal output redacts the subject.

GET /api/admin/cluster asks every peer for private status using the cluster credential, then returns safe fields. An unavailable peer appears with status `unavailable`; one failed peer does not make the entire response fail.

POST /api/admin/test-decisions uses the same body as the public endpoint but authenticates the administrator. The server creates a new request ID if none is supplied and clearly marks the audit actor as admin. It changes bucket state.

8.18 RequestVote RPC

POST /internal/raft/request-vote requires cluster bearer authentication and a configured source node.

Request:

```
{
  "protocol_version": 1,
  "cluster_id": "quorumlimiter-local",
  "source_node_id": "node2",
  "term": 19,
  "candidate_id": "node2",
  "last_log_index": 849,
  "last_log_term": 18
}
```

Response 200:

```
{
  "protocol_version": 1,
  "source_node_id": "node1",
  "term": 19,
  "vote_granted": true
}
```

Malformed or unknown source returns 400 or 403. Invalid cluster token returns 401. Protocol or cluster mismatch returns 409. Internal timeout returns 503. A normal denied vote still returns 200 with `vote_granted: false`.

8.19 AppendEntries RPC

POST `/internal/raft/append-entries` requires cluster authentication.

Request:

```
{
  "protocol_version": 1,
  "cluster_id": "quorumlimiter-local",
  "source_node_id": "node1",
  "term": 19,
  "leader_id": "node1",
  "prev_log_index": 849,
  "prev_log_term": 18,
  "entries": [
    {
      "index": 850,
      "term": 19,
      "kind": "noop",
      "command": null
    }
  ],
  "leader_commit": 849
}
```

Success response:

```
{
  "protocol_version": 1,
  "source_node_id": "node2",
  "term": 19,
  "success": true,
  "match_index": 850,
  "conflict_term": 0,
  "conflict_index": 0
}
```

A log mismatch returns 200, `success: false`, and conflict hints. Authentication, protocol, validation, and internal errors use the same categories as RequestVote.

8.20 Internal status

GET `/internal/raft/status` requires the cluster token. It returns node ID, role, term, leader ID, last log index and term, commit index, last applied, database bytes, last leader contact, last quorum contact, and peer replication progress. It never returns secret values or command payloads.

9. Testing and Quality Checks

9.1 Test strategy

Use a pyramid. Pure deterministic calculations receive many fast unit tests. Storage and HTTP components receive focused integration tests. A smaller set starts three real nodes and injects faults. Manual checks cover browser usability, operating system firewalls, certificate behavior, and visual layout.

No single test proves consensus correctness. Confidence comes from explicit invariants, repeated fault scenarios, race detection, restart recovery, and comparison against a reference model.

9.2 Unit test cases

Area	Test case	Expected result
Config	Exactly three valid peers	Config loads
Config	Duplicate ID or URL	Startup error names the field
Config	Election minimum below three heartbeats	Startup error
Storage	First open and migration	All version 1 buckets exist
Storage	Reopen after term and vote	Exact values recovered
Storage	Append then truncate suffix	Prefix retained, suffix removed
Token bucket	New subject	Starts at full capacity
Token bucket	Five cost 1 requests at capacity 5	Five allowed
Token bucket	Sixth immediate request	Denied
Token bucket	Partial interval	Correct milli tokens and remainder
Token bucket	Backward command time	Effective time does not decrease
Token bucket	Reaches capacity	Clamped and remainder reset
Idempotency	Same key and same body	Original result, no second deduction
Idempotency	Same key and different body	Conflict
Auth	Correct client key	Client resolved
Auth	Wrong suffix with real prefix	Generic failure
Auth	Revoked client	Generic unauthorized
Session	Modified payload	Rejected
Session	Expired session	Rejected and cookie cleared
Vote	Candidate has older last term	Vote denied
Vote	Equal term and higher last index	Vote may be granted
Vote	Already voted for another node	Vote denied
Append	Previous entry mismatch	Failure with conflict hint
Append	Conflicting suffix	Suffix replaced transactionally
Commit	Majority has current term entry	Commit advances
Commit	Majority only has old term entry	Commit does not advance directly
Apply	Commit jumps several indexes	Every command applied in order

9.3 Integration test cases

Scenario	Actions	Expected result
Initial cluster	Start three empty nodes	One leader, same term, no split leadership
Follower outage	Stop one follower and propose	Two nodes commit and apply
Leader outage	Stop leader	New leader in higher term, service resumes
No quorum	Leave one node connected	Mutations fail with 503 or timeout, no commit
Heal follower	Restore isolated follower	Log and state converge
Conflicting old leader	Isolate leader, elect new leader, heal	Old uncommitted suffix is removed
Full restart	Stop all nodes, restart with volumes	State, term, log, and decisions recover
Duplicate across nodes	Send same key through two nodes	One logical effect
Hot bucket concurrency	Send 100 cost 1 calls to capacity 20	At most 20 allowed before refill
Policy update	Change refill and reduce capacity	Update command credits old rate through update time and clamps immediately
Client revocation	Revoke then call public API	Unauthorized on every node
Slow follower	Delay one node messages	Quorum continues, delayed node catches up
Duplicate RPC	Deliver AppendEntries twice	No duplicate log or state effect
Stale response	Delay old term vote or append response	Current state unchanged
Process crash during proposal	Stop leader before or after follower write	Retry resolves once, never charges twice

9.4 Invariant checks

Add helper assertions to repeated integration tests:

- At most one leader exists for a given term among observed nodes.
- Current term, commit index, and last applied never decrease on one persistent node.
- `last_applied <= commit_index <= last_log_index`.
- Every committed index has the same term and command digest on all nodes that contain it.
- A committed entry is never replaced at the same index.
- Nodes at the same applied index have identical normalized application state.
- A decision record's log index matches the audit key and applied entry.
- Token balance always remains between zero and capacity.
- A request ID has one request digest and one result per client.

9.5 API and error handling checks

Test every endpoint from Section 8. For JSON endpoints, include no content type, wrong content type, empty body, malformed JSON, unknown field, duplicate field if the decoder permits detection, excessive body, invalid UTF 8, out of range integer, unauthorized, revoked, expired session, missing CSRF, wrong origin, no leader, no quorum, and proposal timeout.

Verify response content type, request ID, cache headers, security headers, status code, stable error code, and absence of internal stack details. Close response bodies in all Go tests and client code.

9.6 Security checks

Run:

```
go vet ./...
golangci-lint run ./...
govulncheck ./...
git diff --check
git grep -nE \
  '(qlk_[A-Za-z0-9_-]{20,}|QL_(ADMIN|CLUSTER|SESSION|API_KEY)).*=[^<])' \
  -- '!*.example' '!tmp/*'
```

Then perform these manual checks:

- Public Caddy route cannot reach `/internal/raft/*`.
- Public route cannot reach `/metrics`.
- Node debug ports bind to loopback in the single server deployment.
- Session cookie has HTTP only, SameSite Strict, correct Path, and Secure in production.
- Admin responses use `Cache-Control: no-store`.
- Content Security Policy blocks inline scripts unless an intentional nonce is used.
- Login returns the same visible error for unknown and almost correct tokens.
- Logs, metrics, audit rows, error responses, screenshots, and traces contain no secrets.
- An API key is visible only in its one time creation response.
- Large and slow request bodies cannot hold connections indefinitely.
- Dependency and image versions are pinned.

9.7 Race, leak, and shutdown checks

Run:

```
go test -race ./... -count=1 -timeout=10m
go test ./test/integration -count=20 -timeout=20m
```

Capture a goroutine profile before and after 1,000 timed out proposals in a test build. The waiter map and goroutine count should return close to baseline. Start and stop the cluster repeatedly. No test should hang after its context expires.

Use `go.uber.org/goleak` only if you decide the additional test dependency is worth it. It is optional and is not part of the locked core stack.

9.8 Responsive and accessibility checks

Test dashboard pages at 1440, 1024, 768, 390, and 320 CSS pixels. At every width:

- Navigation remains reachable.
- Forms have no clipped labels or buttons.
- Tables scroll inside their container instead of widening the entire page.
- Long IDs wrap or truncate with an accessible full value.
- Status is communicated with text, not color alone.
- Focus is visible.

- Tab order follows reading order.
- Error summary links or focuses the affected field.
- Zoom at 200 percent remains usable.
- Reduced motion preference disables nonessential animation.

Run an automated browser accessibility scanner if available, then manually verify keyboard and screen reader labels. Automated tools do not prove usability.

9.9 Load test cases

Baseline scenario:

- 25 virtual users.
- 60 seconds.
- Unique subject per virtual user and iteration.
- A policy with high enough capacity to avoid expected denial.
- Threshold p95 below 250 milliseconds on the stated development machine.
- Unexpected response rate below 1 percent.

Hot key scenario:

- 50 virtual users.
- One shared subject.
- Capacity 100 and no meaningful refill during the first second.
- Exactly 100 cost 1 decisions allowed before refill, accounting for any deliberate elapsed time.
- Remaining calls receive 429, not 500.

Failover scenario:

- Stable traffic for 20 seconds.
- Stop the current leader.
- Continue for 40 seconds.
- Record time to new leader, temporary 503 count, p95 before and after, and final convergence.

9.10 Continuous integration checks

The CI workflow should use read only repository permissions by default and run:

1. Checkout with pinned action commit.
2. Set up Go 1.26.5.
3. Verify `go mod tidy` produces no diff.
4. Verify formatting.
5. Run `go vet`.
6. Run unit and integration tests.
7. Run race tests.

8. Run golangci-lint 2.12.2.
9. Run govulncheck 1.1.4.
10. Build the binary and Docker image.
11. Start Compose, run smoke test, and tear down without printing `.env`.

Use generated temporary CI secrets. Do not place repository secrets in pull request workflows from forks.

9.11 Final completion checklist

Repository and setup

- [] Go toolchain is 1.26.5 and `go.mod` uses the intended module path.
- [] `bbolt` is 1.5.0 and Prometheus client is 1.24.1.
- [] `.env` is ignored and mode 0600.
- [] `.env.example` contains placeholders only.
- [] A fresh clone can follow README setup.

Token bucket

- [] Token arithmetic uses milli tokens and no floating point.
- [] Refill remainder is preserved.
- [] Effective command time cannot move backward.
- [] Balance cannot fall below zero or exceed capacity.
- [] Duplicate requests do not refill or deduct again.
- [] Policy update behavior is documented and tested.

Raft lite

- [] Term and vote persist before responses.
- [] Vote log freshness compares last term then last index.
- [] Election timeouts are randomized.
- [] Exactly one leader is observed per term.
- [] `AppendEntries` checks previous index and term.
- [] Conflicting suffixes are repaired.
- [] Leader tracks next and match indexes.
- [] Commit requires majority and current term rule.
- [] Apply is ordered and only committed entries are applied.
- [] Restart replays committed unapplied entries.
- [] No quorum produces no successful mutation.

API and security

- [] Public decision endpoint validates headers and JSON.
- [] Follower forwarding cannot loop.

- [] API keys use random values and HMAC digests.
- [] Raw keys are displayed once and never stored.
- [] Revoked clients fail on all nodes.
- [] Admin session signature and expiry are verified.
- [] CSRF and same origin checks protect mutations.
- [] Production cookies are Secure and HTTP only.
- [] Internal routes are private and cluster authenticated.
- [] Secrets and full subjects are absent from logs and metrics.
- [] Body sizes and server timeouts are configured.

Dashboard

- [] All seven screens exist and match documented actions.
- [] Every form has labels, validation, CSRF, and clear feedback.
- [] One time key page uses no store caching.
- [] No destructive cluster controls exist.
- [] Pages work without JavaScript for essential actions.
- [] Layout works at 320 through 1440 pixels.
- [] Keyboard focus and text status are clear.

Testing and operations

- [] Unit, integration, repeated, and race tests pass.
- [] Leader failover script passes.
- [] Conflicting log and restart tests pass.
- [] Load tests record hardware and request mix.
- [] Readiness changes correctly during failures.
- [] Prometheus scrapes all nodes.
- [] Database volumes persist across container recreation.
- [] Backup and restore rehearsal succeeds.
- [] Public HTTPS and firewall checks pass.
- [] CI is green with least privilege.

Documentation

- [] README includes architecture, quick start, API, tests, demo, and deployment.
- [] Limitations clearly state fixed membership and no compaction.
- [] Screenshots contain no secrets or user data.
- [] Benchmarks are reproducible and not exaggerated.
- [] Git history contains no credentials.

10. Deployment

10.1 Deployment choices

There are two supported demonstration layouts:

Layout	What it proves	What it does not prove
Three containers on one Ubuntu server	Process isolation, network RPC, independent disks, election, restart, quorum, and secure public access	Survival of physical server or host network failure
One node on each of three virtual machines	The above plus machine and zone fault tolerance when placed carefully	Dynamic membership, automatic volume replacement, or formally verified production consensus

Start with the single server layout. Move to three machines only after all fault tests pass.

10.2 Required production environment variables

Create `/opt/quorumlimiter/.env` with mode `0600`. Use real generated values only on the server:

```
DOMAIN=limiter.example.com
QL_CLUSTER_ID=quorumlimiter-production
QL_CLUSTER_TOKEN=<GENERATE_WITH_OPENSSL>
QL_ADMIN_TOKEN=<GENERATE_WITH_OPENSSL>
QL_SESSION_KEY=<GENERATE_WITH_OPENSSL>
QL_API_KEY_PEPPER=<GENERATE_WITH_OPENSSL>
QL_PUBLIC_BASE_URL=https://limiter.example.com
QL_PRODUCTION=true
QL_LOG_LEVEL=info
QL_ELECTION_TIMEOUT_MIN=800ms
QL_ELECTION_TIMEOUT_MAX=1400ms
QL_HEARTBEAT_INTERVAL=200ms
QL_PROPOSAL_TIMEOUT=3s
```

Generate each secret independently:

```
umask 077
openssl rand -base64 48
```

Store them in a password manager as well as the protected server file. Never put them in Compose YAML, Dockerfile, Caddyfile, shell history comments, GitHub Actions variables for a public demo, or screenshots.

10.3 Production image

The Dockerfile should use:

```
FROM golang:1.26.5-alpine3.23 AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
ARG VERSION=dev
RUN CGO_ENABLED=0 GOOS=linux go build \
    -trimpath \
    -ldflags="-s -w -X main.version=${VERSION}" \
    -o /out/quorumlimiter ./cmd/quorumlimiter

FROM alpine:3.23
RUN addgroup -S -g 10001 app && \
    adduser -S -D -H -u 10001 -G app app && \
    apk add --no-cache ca-certificates tzdata && \
    install -d -o app -g app -m 0700 /data
COPY --from=build /out/quorumlimiter /usr/local/bin/quorumlimiter
USER 10001:10001
EXPOSE 8080
ENTRYPOINT ["/usr/local/bin/quorumlimiter"]
```

Build and inspect locally:

```
docker build \
    --build-arg VERSION=v1.0.0 \
    -t ghcr.io/yourname/quorumlimiter:v1.0.0 .
docker image inspect ghcr.io/yourname/quorumlimiter:v1.0.0 \
    --format '{{.Config.User}} {{.Size}}'
```

Publishing the image is optional. If used, authenticate through your configured GitHub or container registry credential helper. Do not paste a token into a command shown to an AI agent.

10.4 Single server Compose design

The production Compose file should define:

- node1, node2, and node3 using the same pinned application image.
- One named volume per node.
- One private network.
- Caddy 2.11.4 as the only service publishing 80:80 and 443:443.
- Prometheus 3.13.1 with no public port, or loopback 127.0.0.1:9090:9090.
- Health checks against each service's local readiness route.
- Secrets read from the protected environment file.

The peer value inside Compose is:

```
node1=http://node1:8080,node2=http://node2:8080,node3=http://node3:8080
```

Each node has its own QL_NODE_ID, QL_ADVERTISE_URL, and data volume. Everything else is shared.

10.5 Caddy production configuration

A minimal Caddyfile:

```
{ $DOMAIN } {
    encode zstd gzip

    @private path /internal/* /metrics
    respond @private 404

    reverse_proxy node1:8080 node2:8080 node3:8080 {
        lb_policy random_choose 2
        health_uri /health/live
        health_interval 10s
        fail_duration 10s
    }

    header {
        Strict-Transport-Security "max-age=31536000; includeSubDomains"
        X-Content-Type-Options "nosniff"
        Referrer-Policy "no-referrer"
        -Server
    }
}
```

The application still sets its own content security policy, frame policy, and admin cache headers. Test Caddy configuration:

```
docker run --rm \
-v "$PWD/deploy/Caddyfile:/etc/caddy/Caddyfile:ro" \
caddy:2.11.4-alpine \
caddy validate --config /etc/caddy/Caddyfile
```

10.6 Server provisioning

1. Create an Ubuntu server and add your SSH public key.
2. Point the DNS record to its public IP.
3. Install Docker using Section 4.5.
4. Create the deployment directory.
5. Copy `deploy`, the protected `.env`, and either pull or load the pinned image.
6. Enable the firewall.
7. Start Compose.

Commands:

```
sudo install -d -m 0750 -o "$USER" -g "$USER" /opt/quorumlimiter
cd /opt/quorumlimiter
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo ufw status verbose

docker compose -f deploy/compose.prod.yml config
docker compose -f deploy/compose.prod.yml pull
docker compose -f deploy/compose.prod.yml up -d
docker compose -f deploy/compose.prod.yml ps
```

Do not use `docker compose down --volumes` in production.

10.7 First deployment verification

On the server:

```
docker compose -f deploy/compose.prod.yml logs --tail=200
docker compose -f deploy/compose.prod.yml exec node1 \
  wget -qO- http://127.0.0.1:8080/health/live
docker compose -f deploy/compose.prod.yml exec prometheus \
  wget -qO- http://127.0.0.1:9090/-/ready
```

From another computer:

```
curl -fsS https://limiter.example.com/health/live
curl -fsS https://limiter.example.com/health/ready
curl -i https://limiter.example.com/internal/raft/status
curl -i https://limiter.example.com/metrics
curl -I https://limiter.example.com/admin/login
```

Expected public internal and metrics responses are 404 or 403. Open the admin page, create a production test policy and client, store the raw key safely, send allowed and denied decisions, revoke the test client, and confirm it stops working.

10.8 Production security checks

- Run `ss -lntp` and confirm only intended public ports listen on all interfaces.
- Run `docker compose config` and confirm no literal secret was accidentally placed in the YAML.
- Inspect containers with `docker inspect` and confirm nonroot user, read only root where configured, dropped capabilities, and explicit volumes.
- Confirm admin cookies have Secure, HTTP only, and SameSite Strict.
- Confirm HTTP redirects to HTTPS.
- Confirm HSTS only after HTTPS works reliably.
- Confirm Caddy logs and application logs redact credentials.
- Keep the operating system and Docker security updates current.
- Do not expose Prometheus publicly without authentication.

10.9 Back up a follower without losing quorum

Back up one caught up follower at a time. The example assumes the Compose project is named `quorumlimiter` and `node3` uses volume `quorumlimiter_node3_data`. Verify actual names with `docker volume ls`.

```
cd /opt/quorumlimiter
mkdir -p backups
chmod 700 backups

docker compose -f deploy/compose.prod.yml stop node3
docker run --rm \
  -v quorumlimiter_node3_data:/source:ro \
  -v "$PWD/backups:/backup" \
  alpine:3.23 \
  sh -c 'cd /source && tar -czf /backup/node3-$(date +%Y%m%dT%H%M%S).tgz .'
docker compose -f deploy/compose.prod.yml start node3
```

Check `node3` returns ready and catches up. Then back up another follower if desired. Encrypt backup archives before moving them off the server because they contain client digests, subjects, and audit data.

10.10 Restore rehearsal

Never test restore against the only production cluster. Create a separate server or local Compose project:

1. Stop the target test node.
2. Create a new empty test volume.
3. Extract the archive into that volume.
4. Start the node with the same node ID and cluster configuration.
5. Confirm it recovers, joins as follower, catches up, and serves matching state.

Example for a test project:

```
docker volume create ql_restore_node3
docker run --rm \
  -v ql_restore_node3:/restore \
  -v "$PWD/backups:/backup:ro" \
  alpine:3.23 \
  sh -c 'cd /restore && tar -xzf /backup/<BACKUP_FILE>.tgz'
```

Use a test Compose override that mounts `ql_restore_node3` into `node3`. Do not overwrite a production volume until a recovery plan has been reviewed.

10.11 Upgrade and rollback

Upgrade one follower at a time, then the remaining follower, then the leader. Before starting, confirm protocol and schema compatibility, take a backup, and record current image digest.

```
docker compose -f deploy/compose.prod.yml pull node2
docker compose -f deploy/compose.prod.yml up -d --no-deps node2
docker compose -f deploy/compose.prod.yml ps node2
```

Wait for `node2` to become ready and catch up. Repeat for `node3`. Finally update the leader, expecting an election. Rollback means restoring the previous image tag in Compose and recreating the affected container without deleting its volume. Never roll back across an incompatible irreversible database migration.

10.12 Three virtual machine variant

Use three Ubuntu servers with stable private addresses, for example:

```
node1 = http://10.20.0.11:8080
node2 = http://10.20.0.12:8080
node3 = http://10.20.0.13:8080
```

Run one application node and one persistent volume on each machine. Allow port 8080 only among those three private addresses. Put Caddy on a fourth small gateway or on `node1` with a health checked upstream list. Public firewall permits only 80 and 443 to the gateway. SSH should be limited to trusted source addresses where practical.

Every node receives the same cluster ID and secrets but a unique node ID and advertised private URL. Prometheus may run on the gateway and scrape private addresses. Use encrypted transport such as a private virtual network, WireGuard, or future mutual TLS if the provider network is not trusted.

The fixed membership limitation remains. If one machine is permanently replaced, use the same node ID only after ensuring the old machine cannot return and the replacement has the correct restored or fresh follower data procedure. Two live machines with the same node ID are unsafe.

10.13 Live acceptance checklist

- [] DNS points to the expected gateway.

- [] HTTPS certificate is valid and renews automatically.
- [] Only ports 22, 80, and 443 are public.
- [] Internal Raft and metrics routes are blocked publicly.
- [] Exactly one leader exists.
- [] All nodes show matching commit and applied indexes after idle convergence.
- [] Policy and client creation commit successfully.
- [] Public decision returns 200 and 429 as expected.
- [] Duplicate request returns the original result.
- [] Revoked key returns 401.
- [] Stopping the leader causes a successful new election.
- [] Restarted node catches up.
- [] Volumes survive image recreation.
- [] Prometheus scrapes every node.
- [] Backup exists, is encrypted off host, and restore was rehearsed.
- [] Logs and screenshots contain no secrets.

11. GitHub Documentation

11.1 Repository organization

Keep generated binaries, .env, database files, coverage output, temporary certificates, and local backups out of Git. Commit source, tests, migration code, safe deployment templates, documentation, diagrams, and redacted screenshots.

Use focused commits that tell the development story:

```
chore: scaffold validated node configuration
feat(storage): persist raft term vote and log
feat(limiter): add deterministic token bucket state machine
feat(raft): implement leader election
feat(raft): replicate and commit log entries
feat(api): add idempotent decision endpoint
feat(admin): secure policy and client management
feat(ui): add operator dashboard
test(cluster): cover partition and restart recovery
ops: add compose prometheus and caddy
docs: add architecture demo and limitations
```

Do not rewrite clear public history merely to make it look as though the project appeared in one commit. The sequence is useful evidence of engineering work.

11.2 Recommended README structure

Title and one sentence

State what the system does and its defining technical choice:

```
QuorumLimiter is a three node distributed token bucket rate limiter whose
state changes are ordered by a Raft lite consensus implementation written in Go.
```

Status badges

Include CI, Go version, license, and latest release only after they exist. Do not add a coverage percentage badge unless coverage is measured in CI and useful.

Demonstration

Place a short GIF or image of leader failover near the top, followed by a two minute script:

1. Start the cluster.
2. Show the leader and matching indexes.
3. Send decisions until one is denied.
4. Stop the leader.
5. Show a new leader and continued request.
6. Restart the old leader and show catch up.

Why this project exists

Explain the multi node over allowance problem. A concrete example with five tokens and three independent servers is more effective than a broad claim about scalability.

Features

Separate application features, consensus features, security, observability, and testing. Keep limitations visible rather than hiding them in the roadmap.

Architecture

Include the system diagram and request sequence. Explain leader forwarding, quorum commit, deterministic apply, and bbolt per node. Link to [docs/architecture.md](#) and the architecture decision records.

Consistency behavior

State:

- Writes require a leader and quorum.
- The service prefers consistency over write availability during a partition.
- A timeout may hide a later commit, so callers retry with the same idempotency key.
- Reads in the dashboard are operational snapshots, not a general linearizable read API.

Quick start

List prerequisites, secret generation, exact Compose command, health check, admin URL, smoke test, failover test, and cleanup without volume deletion.

API example

Use placeholders only. Include both allowed and denied responses plus retry instructions.

Tests

List format, unit, integration, race, lint, vulnerability, smoke, failover, and k6 commands. Link each important distributed test to the invariant it checks.

Metrics and operations

Document readiness states, important metrics, initial Prometheus queries, backup, and logs.

Deployment

Link to the single server and three virtual machine instructions. Put the live URL only if it is safe to expose and maintained.

Design decisions

Summarize why Go, bbolt, server rendered UI, token integer units, fixed membership, and no Redis were chosen.

Limitations

List fixed membership, no snapshots or compaction, one write leader, educational security between nodes, single host demo limitation, and lack of formal verification.

Roadmap, contributing, and license

Make the roadmap match Section 13. Explain how to run checks before a pull request. Choose a license you understand, such as MIT or Apache 2.0.

11.3 Screenshot plan

Recommended images:

Image	What it should prove	What to hide
Overview	One leader, three healthy nodes, request summary	Public IPs and sensitive domains if needed
Policies	Concrete token bucket configuration	Nothing secret should appear
New client confirmation	One time key behavior	Blur the complete raw key
Decisions	Allowed and denied audit rows	Full subjects and client identifiers
Cluster before failure	Matching commit and apply indexes	Private addresses if desired
Cluster after failure	New leader and higher term	No tokens or cookies
Prometheus	Election and decision metrics	Query history containing sensitive labels
CI	Passing tests and security gates	Repository secrets are never visible

Capture at 1440 by 900 or a similarly readable size. Crop browser extensions, password manager popups, bookmarks, terminal prompts, and unrelated tabs. Use synthetic subjects such as `demo_user_0042`.

11.4 CI workflow outline

The workflow should pin action commits rather than floating tags. The exact commit hashes need periodic review, so the repository should use a dependency update tool or a documented manual review.

Conceptual workflow:

```

name: ci

on:
  push:
  pull_request:

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 20
    steps:
      - uses: actions/checkout@<PINNED_COMMIT>
      - uses: actions/setup-go@<PINNED_COMMIT>
        with:
          go-version: 1.26.5
          cache: true
      - run: go fmt ./...
      - run: git diff --exit-code -- '*.go'
      - run: go mod tidy && git diff --exit-code
      - run: go vet ./...
      - run: go test ./... -count=1
      - run: go test -race ./... -count=1 -timeout=10m
      - run: go install github.com/golangci/golangci-lint/v2/cmd/golangci-lint@v2.12.2
      - run: golangci-lint run ./...
      - run: go install golang.org/x/vuln/cmd/govulncheck@v1.1.4
      - run: govulncheck ./...

```

Add a separate Compose smoke job only after the unit and integration job is stable. Generate temporary random secrets inside that job and mask them before any log output.

11.5 Publishing safely

Before creating a remote:

```

git status --short
git diff --check
git ls-files | sort
git grep -nE \
  '(qlk_[A-Za-z0-9_-]{20,}|QL_(ADMIN|CLUSTER|SESSION|API_KEY)).*=[^<])' \
  -- '!*.example'

```

Review every tracked file. Create the empty repository yourself. Use SSH or a system credential manager. Never send a GitHub password, personal access token, SSH private key, recovery code, or session cookie to Claude Code, Codex, ChatGPT, an issue, or a commit.

11.6 Reviewer oriented documentation

Add `docs/demo.md` with exact commands and expected observations. Add `docs/architecture.md` with invariants and failure behavior. Add `docs/security.md` with threat boundaries, secrets, public routes, and known limitations. Architecture decision records should explain alternatives, not repeat implementation.

For interviews, prepare to answer:

- Why must the leader wait for apply rather than only replication?
- Why is the timestamp part of the command?
- What prevents two votes in one term after restart?
- Why can an isolated old leader not commit?
- How does a follower repair a conflicting suffix?
- Why can a timed out client retry safely?

- What breaks if idempotency records are pruned independently?
- What would be required before production use?

12. Troubleshooting Guide

12.1 Installation and environment

Problem	Likely cause	Resolution
go: command not found	Go path not loaded	Export <code>/usr/local/go/bin</code> and Go bin path, then reopen terminal
Go shows wrong version	Older package manager Go appears first	Run <code>which go</code> , remove or reorder the old path, and verify 1.26.5
Go checksum is empty	Official JSON query did not match filename	Check architecture, version, network, and jq; do not install unverified archive
Docker permission denied	User group change not active	Sign out and in, or run <code>newgrp docker</code> , then verify group membership
Docker daemon unavailable	Service is stopped	Run <code>sudo systemctl enable --now docker</code> and inspect service status
Compose version differs	Package repository has a newer patch	Use supported newer patch, validate config, and record actual version
Tool installed but not found	Go bin directory absent from path	Export <code>\$(go env GOPATH)/bin</code> and update profile

12.2 Configuration and startup

Problem	Likely cause	Resolution
Missing required variable	<code>.env</code> was not loaded or name differs	Compare with <code>.env.example</code> , load with <code>set -a; . ./.env; set +a</code>
Duplicate peer error	Same node ID or URL appears twice	Correct <code>QL_PEERS</code> and keep exactly node1, node2, node3
Local node absent	<code>QL_NODE_ID</code> does not match peer map	Make the local ID exactly one peer key
Invalid timeout relationship	Election minimum is too close to heartbeat	Use 200 ms heartbeat and 800 to 1400 ms election range
bbolt open timeout	Another process owns the same file	Stop duplicate node or give every node a distinct data path
Permission denied on database	Parent or volume ownership is wrong	Create directory mode 0700 and make runtime user 10001 its owner
New binary rejects schema	Database version is newer	Restore compatible binary or perform a reviewed migration, never force version backward

12.3 Leader election

Problem	Likely cause	Resolution
No leader ever appears	Peer URLs fail, token mismatch, or timer not firing	Curl private health, inspect RPC counters, compare cluster token and peer names
Elections repeat constantly	Heartbeats fail or election timer resets incorrectly	Check AppendEntries auth and reset timer only on valid current leader calls
Repeated split votes	Identical random seed or narrow timeout	Use independent random sources and the full configured range
Node votes twice after restart	Vote not persisted before response	Move persistence before returning the granted vote and add restart test
Stale node wins	Up to date log comparison is wrong	Compare last log term first, then last index
Old leader remains leader	Higher term response ignored	Every RPC path must step down and persist when it sees a higher term

12.4 Replication and commit

Problem	Likely cause	Resolution
Follower never catches up	next index retry or conflict hint is wrong	Log safe index and term metadata, then test shorter and conflicting logs
Duplicate entries appear	Follower appends without matching and truncating	Verify previous entry, remove conflicting suffix, then append transactionally
Commit advances with one node	Leader counts itself incorrectly or quorum is wrong	In three nodes require two match indexes including leader
Old term entry commits unsafely	Current term commit rule omitted	Only advance directly to an index whose entry term equals current term
Apply index skips	Apply loop uses latest only	Iterate every index from last applied plus one through commit
Proposal returns before state exists	Waiter completes at append or commit	Complete only after deterministic apply result is available
Waiter count grows	Timeout or shutdown does not clean registry	Remove waiters on result, timeout, cancellation, and node stop

12.5 Token bucket and idempotency

Problem	Likely cause	Resolution
Nodes have different balances	Local clocks or floating point used	Apply only command timestamp and integer milli token arithmetic
Refill is slowly too low	Division remainder discarded	Store numerator remainder in bucket state
Refill exceeds capacity	Missing clamp or stale remainder	Clamp and reset remainder when full
Time moves backward	New leader clock is behind	Use maximum of command timestamp and last refill
Duplicate charges twice	Idempotency lookup occurs too late	Return stored result before refill and deduction
Same key hides different body	Request digest not checked	Store canonical request digest and return 409 on mismatch
Negative balance	Cost or arithmetic not validated	Validate cost, use checked math, and assert state bounds

12.6 API and authentication

Problem	Likely cause	Resolution
Valid key returns 401	Prefix extraction, pepper, or digest differs	Confirm shared pepper and exact raw key parsing without logging the key
Key works on one node only	Client state not committed or pepper differs	Compare applied indexes and shared secret configuration
Follower returns redirect loop	Forward hop marker missing	Add one hop maximum and leader ID validation
504 appears during load	Proposal or quorum latency exceeds timeout	Inspect disk, apply lag, RPC latency, and retry same idempotency key
Client creation returns 504	The digest may commit after the one time raw key is lost	List clients, revoke the unusable committed client if present, then create another
Admin session fails after load balancing	Session key differs across nodes	Use one shared <code>QL_SESSION_KEY</code>
CSRF fails behind Caddy	Origin reconstruction uses internal HTTP scheme	Honor trusted forwarded protocol from Caddy only
Browser shows insecure cookie	Production flag or HTTPS base URL is wrong	Set production true and use the real HTTPS public URL
JSON unknown fields accepted	Decoder lacks strict mode	Call <code>DisallowUnknownFields</code> and ensure a single JSON value

12.7 Dashboard

Problem	Likely cause	Resolution
Templates missing in container	Files were not embedded or path is wrong	Use <code>go:embed</code> , build the binary, and test image directly
Static files return 404	Route or embed subdirectory mismatch	List embedded paths in a test and align route stripping
One time key appears after refresh	Raw key stored in session or database	Render only from immediate creation response and use post confirmation flow
User text renders as HTML	Unsafe template type conversion	Pass plain strings and let html/template escape
Mobile page has horizontal overflow	Long table or identifier controls page width	Wrap IDs and put overflow on table container
Button works only with JavaScript	Core action implemented as client script	Add a normal server form, then enhance optionally

12.8 Docker and networking

Problem	Likely cause	Resolution
Node cannot reach peer	Peer URL uses localhost inside container	Use Compose service name and container port
All nodes show same data file lock	They share a volume	Assign one named volume to each node
Caddy returns 502	Nodes are unhealthy or upstream names wrong	Inspect Compose health and Caddy upstream service names
Public internal route is reachable	Caddy matcher order is wrong	Put explicit private path response before reverse proxy and test it
Prometheus target down	Wrong scrape name, port, or metrics route	Curl node metrics from Prometheus container
Data disappears after recreate	Anonymous volume or accidental volume deletion	Use named volumes and never run down with volumes
Health remains starting	Readiness requires leader and cluster cannot elect	Diagnose election rather than weakening health check

12.9 Deployment and HTTPS

Problem	Likely cause	Resolution
Certificate issuance fails	DNS, firewall, or port conflict	Confirm DNS public IP, open 80 and 443, and stop conflicting web server
HTTPS redirect loop	Proxy headers or public URL disagree	Trust Caddy headers and set one HTTPS base URL
Domain works intermittently	DNS has old addresses or one upstream is broken	Inspect DNS records and each Caddy upstream
Server disk grows quickly	No log compaction plus audits and metrics	Monitor volume, prune audits deterministically, back up, and plan snapshot feature
Backup archive is empty	Wrong Compose volume name	Inspect mounts and archive listing before considering backup complete
Restored node refuses cluster	Wrong node ID, cluster ID, or newer schema	Match identity and version, then restore into an isolated test cluster first

12.10 Tests and CI

Problem	Likely cause	Resolution
Integration test flakes	Fixed sleeps or shared ports	Use free listeners, bounded polling, isolated temp directories, and diagnostics
Race test hangs	Goroutine does not observe cancellation	Audit channel sends, timers, response bodies, and transport contexts
CI says go.mod changed	Dependencies were not tidied locally	Run <code>go mod tidy</code> , inspect diff, and commit intended changes
Linters differs locally	Wrong golangci-lint version	Install and verify 2.12.2
k6 cannot reach host	Container networking differs	Use host network on Linux or <code>host.docker.internal</code> elsewhere
k6 shows many 429 responses	Policy intentionally exhausted	Separate expected denial from unexpected errors
govulncheck reports issue	Reachable vulnerable dependency	Upgrade safely, test, and document the fix

13. Future Improvements

13.1 Essential requirements for version 1.0

These are not optional. The project should not be called complete without them:

- Deterministic token bucket with integer arithmetic.
- Durable term, vote, log, and applied state.
- Leader election, log replication, majority commit, and current term rule.
- Fixed three node configuration.
- Safe idempotent decision API.
- Admin policy and client lifecycle with one time raw keys.
- Secure sessions, CSRF, private internal routes, timeouts, and redaction.
- Full dashboard, health, readiness, metrics, and structured logs.
- Restart, failover, partition, conflicting log, race, and load tests.
- Reproducible Docker deployment, persistence, backup, and documentation.

13.2 High value next improvements

Snapshot and log compaction

Create a deterministic snapshot of application state at a committed index, persist snapshot metadata, implement InstallSnapshot for lagging followers, and discard old log prefixes safely. This addresses the largest operational limitation. It is also a substantial consensus change and needs its own fault tests.

Linearizable read path

Add a read index or quorum confirmation method for API reads that must observe the latest committed write. Current dashboard reads are operational snapshots. Do not claim general linearizable reads until the algorithm and tests exist.

Mutual TLS between nodes

Replace the shared internal bearer token as the primary network identity with node certificates, mutual verification, rotation, and private names. Keep authorization by configured node ID.

Clock monitoring

Expose clock offset monitoring and refuse leadership when offset exceeds a documented threshold. The command timestamp approach keeps nodes deterministic, but a badly wrong leader clock can affect refill fairness.

Snapshot aware backup

Produce an application coordinated snapshot and manifest containing cluster ID, node ID, applied index, checksum, schema version, and software version. Verify checksums during restore.

13.3 Advanced distributed systems extensions

Dynamic membership

Implement joint consensus for safe addition or removal of nodes. This is not simply editing the peer list. Old and new majorities must overlap during transition.

Pre vote and leadership transfer

Pre vote can reduce disruptive term increases from an isolated node. Leadership transfer can support planned maintenance. Both require careful tests around stale and unavailable targets.

Read leases

A leader lease can reduce quorum reads, but only under explicit timing assumptions. Incorrect lease handling can return stale results, so this should follow clock monitoring and linearizable read tests.

Batched proposals and group commit

Batch compatible log appends and bbolt flushes to improve throughput. Preserve per request idempotency and ordered results. Measure the latency tradeoff rather than assuming batching is better.

Sharding

Partition policies and subjects across multiple independent Raft groups. This increases write throughput but adds routing, rebalancing, multi group operations, and much more operational complexity.

13.4 Product improvements

- Policy namespaces and tenant isolation.
- Scoped admin roles rather than one shared administrator.
- API key rotation workflow with overlapping validity.
- OpenAPI 3.1 specification and generated clients.
- Webhook or alert integration for quorum loss and unusual denial rates.
- Exportable audit reports with retention controls.
- Per policy charts in the dashboard without high cardinality metric labels.
- Configurable idempotency and audit retention with deterministic pruning commands.
- A command line client for decisions and cluster diagnostics.
- Dark theme after accessibility and contrast testing.

13.5 Research and verification improvements

Model the core election and log rules in TLA+ or another specification language, then use model checking to search small state spaces for safety violations. Add property based tests that generate message loss, duplication, and reordering. Compare trace invariants with the executable implementation.

This work would materially strengthen the project. It also encourages a useful interview stance: tests provide evidence, while a small educational implementation is not automatically proven correct.

Appendix A. Command Reference

A.1 Daily development

```
make fmt
make test
make race
make lint
make vuln
```

A.2 Local cluster

```
docker compose -f deploy/compose.dev.yml up -d --build
docker compose -f deploy/compose.dev.yml ps
docker compose -f deploy/compose.dev.yml logs -f --tail=100
make smoke
make failover
docker compose -f deploy/compose.dev.yml down
```

A.3 Diagnose nodes

```
curl -fsS http://127.0.0.1:18081/health/ready | jq
curl -fsS http://127.0.0.1:18082/health/ready | jq
curl -fsS http://127.0.0.1:18083/health/ready | jq
docker compose -f deploy/compose.dev.yml logs node1 --tail=200
```

A.4 Final release check

```
go fmt ./...
git diff --exit-code -- '*.go'
go mod tidy
git diff --exit-code -- go.mod go.sum
go vet ./...
go test ./... -count=1
go test -race ./... -count=1 -timeout=10m
golangci-lint run ./...
govulncheck ./...
docker compose -f deploy/compose.dev.yml config
```

Appendix B. Environment Variable Reference

Variable	Example placeholder	Required	Purpose
QL_NODE_ID	node1	Yes	Unique fixed node identity
QL_CLUSTER_ID	quorumlimiter-local	Yes	Prevents cross cluster RPC acceptance
QL_BIND_ADDR	:8080	Yes	Node HTTP listener
QL_ADVERTISE_URL	http://node1:8080	Yes	Address peers use
QL_PEERS	node1=http://node1:8080,...	Yes	Fixed three node membership
QL_DATA_PATH	/data/quorumlimiter.db	Yes	Local bbolt file
QL_CLUSTER_TOKEN	<RANDOM_SECRET>	Yes	Internal RPC bearer credential
QL_ADMIN_TOKEN	<RANDOM_SECRET>	Yes	Administrator sign in secret
QL_SESSION_KEY	<RANDOM_SECRET>	Yes	Admin cookie signature
QL_API_KEY_PEPPER	<RANDOM_SECRET>	Yes	HMAC key for client key digests
QL_ELECTION_TIMEOUT_MIN	800ms	Yes	Earliest election timeout

Variable	Example placeholder	Required	Purpose
QL_ELECTION_TIMEOUT_MAX	1400ms	Yes	Latest election timeout
QL_HEARTBEAT_INTERVAL	200ms	Yes	Leader heartbeat period
QL_PROPOSAL_TIMEOUT	3s	Yes	HTTP wait for apply result
QL_PUBLIC_BASE_URL	http://localhost:8080	Yes	Origin and secure redirect basis
QL_PRODUCTION	false	Yes	Enables production cookie and proxy rules
QL_LOG_LEVEL	info	No	Structured log threshold
QL_REVEAL_SUBJECTS	false	No	Development only audit display override

Appendix C. Interview Demonstration Script

1. Explain the over allowance problem using three independent five token servers.
2. Show the architecture diagram and point to one leader plus two followers.
3. Create a five token policy and a client.
4. Send six immediate decisions. Show five 200 responses and one 429.
5. Repeat an allowed request ID and show `duplicate: true` with no extra deduction.
6. Show matching commit and applied indexes.
7. Stop the leader and time the new election.
8. Send another decision through the same Caddy address.
9. Restart the old leader and show it becomes a follower and catches up.
10. Explain why the isolated old leader could not commit, why a timeout requires the same idempotency key, and why the project is still called Raft lite.

Keep the demonstration under five minutes. Have test output and architecture details ready for deeper questions.

Appendix D. Official References and Version Notes

The guide's version lock was checked on July 29, 2026.

- [Go release history](#) lists Go 1.26.5 as the July 2026 patch release.
- [Go installation documentation](#) explains official installation and verification.
- [Raft extended paper](#) is the primary algorithm reference.
- [USENIX Raft paper page](#) provides the publication record and open access material.
- [bbolt releases](#) lists v1.5.0.
- [Prometheus Go client releases](#) lists v1.24.1.
- [Prometheus downloads](#) lists Prometheus 3.13.1 LTS.

- [Prometheus Go instrumentation guide](#) describes the official client.
- [Docker Engine for Ubuntu](#) provides official repository installation.
- [Docker Compose release notes](#) list Compose 5.3.0.
- [Caddy releases](#) list Caddy 2.11.4.
- [Caddy automatic HTTPS](#) explains certificate provisioning and renewal.
- [k6 releases](#) list k6 2.1.0.
- [golangci-lint releases](#) list 2.12.2.
- [Go vulnerability management](#) explains govulncheck.
- [govulncheck releases](#) list v1.1.4.

Before a future upgrade, read release notes, change one dependency, run the complete release gate, repeat fault tests, and record the result. Do not silently mix versions across commands, Docker images, CI, and documentation.

Appendix E. Final Handoff

The project guide is successful if a motivated beginner can take one phase prompt, let an AI coding agent implement it, inspect the exact files, run the stated tests, recognize the expected output, and diagnose common failures before moving on.

The most important habit is pacing. Consensus code can appear convincing while hiding a single unsafe transition. Keep phases small, preserve test evidence, and be willing to stop when an invariant is unclear. A smaller system you can explain is a much stronger portfolio project than a larger one assembled without understanding.